



**UTM**  
UNIVERSITI TEKNOLOGI MALAYSIA

Malaysia-Japan  
International  
Institute of Technology  
(MJIIT)

UNIVERSITI TEKNOLOGI MALAYSIA

FINAL INTERGRATED DESIGN PROJECT REPORT

SESSION 2025/2026

PROJECT TITLE : PLUG AND PLAY HYBRID SENSOR FOR  
SOLAR TRACKING

GROUP MEMBERS : SIM HORNG VEI (A23MJ0111)  
: IRFAN HAKIMI BIN ZULKIFLI (A23MJ0054)  
: ARGYAN RIDHO AL GHIFARI (A23MJ8010)  
: ADIL AKID BIN ABD HAMID (A23MJ0021)

COURSE NAME : INTERGRATED DESIGN PROJECT

COURSE CODE : SMJE 3093

INSTRUCTORS : 1. DR MOHD FITRI BIN MOHD YAKUB  
2. DR MOHD AZIZI BIN ABDUL RAHMAN

INDUSTRY PARTNER : MR. FIRDAUS HAKIMI

## ABSTRACT

The continuous global transition toward renewable energy necessitates drastic improvements in the thermodynamic and photoelectric efficiency of photovoltaic (PV) arrays. While dual-axis tracking systems are mathematically proven to yield up to a 40% increase in energy capture compared to static arrays, traditional commercial solutions are critically flawed. They are prohibitively expensive, rely on complex, failure-prone hardwired harnesses, and lack intelligent environmental awareness, making them unsuitable for retroactive installation on existing infrastructure.

This project details the engineering, deployment, and validation of the "Soltra" (formerly Sensol) ecosystem during its Monozukuri phase. Soltra is a radically decentralized, wirelessly meshed dual-axis solar tracking retrofit designed for extreme reliability, ease of deployment, and high-fidelity cloud telemetry. Overcoming the physical limitations of early Monozukuri concepts which relied on noise-susceptible 2-meter analog cables, the system architecture was entirely revolutionized. The final production prototype features a peer-to-peer wireless mesh network operating on the ultra-low latency ESP-NOW protocol. Four independent, battery-powered Sseeed Studio XIAO ESP32C3 edge nodes act as the sensory extremities, capturing precise lux and infrared data via TSL2591 sensors, alongside localized UV index and battery telemetry.

Central to this architecture is the Heltec WiFi LoRa 32 V3 (ESP32-S3) Master Hub, which processes the massive influx of wireless telemetry using a Prioritized Finite State Machine (PFSM) governed by FreeRTOS. To eliminate the risk of tracking failure due to sensor blinding, the hub executes a zero-dependency, inline Ephemeris mathematical fallback algorithm, while a dedicated ESP32-S3-CAM node provides an overriding Computer Vision tracking layer and live MJPEG video stream. Furthermore, actuation logic is entirely air-gapped; a dedicated ESP32 wireless motor controller handles dual 12V linear actuators, incorporating a 900ms debouncing algorithm to isolate high-voltage back-EMF and prevent mechanical runaway.

Beyond physical hardware, the IDP encompasses a sprawling, enterprise-grade cloud architecture. The Master Hub bridges local telemetry to a Next.js 15 App Router SaaS platform via HiveMQ Serverless MQTT and direct HTTP POST ingestion into a Supabase PostgreSQL database. This allows for persistent historical storage, Row-Level Security (RLS), and granular datasheet exports. A custom Mobile HUD application serves as the primary on-site operator interface, evolving from early desktop sandbox experiments. Finally, the system pioneers the integration of Edge Artificial Intelligence, utilizing Roboflow object detection models with a Gemini AI fallback to classify overhead weather patterns, dynamically overriding hardware tracking behaviours during heavy overcast conditions to minimize parasitic motor draw. The culmination of these systems presents a fully realized, commercial-ready IoT ecosystem that redefines the retroactive solar tracking market.

## TABLE OF CONTENTS

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>ABSTRACT .....</b>                                                  | <b>2</b>  |
| <b>TABLE OF CONTENTS .....</b>                                         | <b>3</b>  |
| <b>LIST OF TABLES .....</b>                                            | <b>6</b>  |
| <b>LIST OF FIGURES .....</b>                                           | <b>7</b>  |
| <b>LIST OF ABBREVIATIONS .....</b>                                     | <b>8</b>  |
| <b>CHAPTER 1 .....</b>                                                 | <b>9</b>  |
| <b>INTRODUCTION.....</b>                                               | <b>9</b>  |
| <b>1.2 OBJECTIVES .....</b>                                            | <b>10</b> |
| <b>1.3 PROJECT SIGNIFICANCE &amp; SCOPE.....</b>                       | <b>11</b> |
| <b>1.4 DISTRIBUTION OF WORKLOAD .....</b>                              | <b>12</b> |
| <b>1.5 PROJECT MILESTONE .....</b>                                     | <b>14</b> |
| <b>CHAPTER 2 .....</b>                                                 | <b>16</b> |
| <b>2.1 BACKGROUND.....</b>                                             | <b>16</b> |
| <b>2.1.1 The Physics of Solar Tracking and Cosine Losses.....</b>      | <b>16</b> |
| <b>2.1.2 Traditional Sensory Topologies: The Flaws of the LDR.....</b> | <b>16</b> |
| <b>2.1.3 The Advent of Decentralized IoT and Edge Computing.....</b>   | <b>17</b> |
| <b>2.1.4 Edge AI and Computer Vision in Solar Systems.....</b>         | <b>17</b> |
| <b>2.2 MARKET ANALYSIS .....</b>                                       | <b>18</b> |
| <b>2.2.1 Utility-Scale Commercial Trackers (e.g., NEXTracker).....</b> | <b>18</b> |
| <b>2.2.2 Consumer-Grade DIY Kits (e.g., Eco-Worthy).....</b>           | <b>18</b> |
| <b>2.2.3 The Soltra Market Positioning.....</b>                        | <b>18</b> |
| <b>CHAPTER 3 .....</b>                                                 | <b>19</b> |
| <b>3.1 HARDWARE ARCHITECTURE &amp; WIRELESS EDGE NETWORK.....</b>      | <b>19</b> |
| <b>3.1.1 The Shift to the Wireless XIAO ESP32C3 Mesh.....</b>          | <b>19</b> |
| <b>3.1.2 Custom PCB Fabrication (soltra-sensor-node-pcb).....</b>      | <b>20</b> |
| <b>3.1.3 The Heltec V3 ESP32-S3 Master Hub .....</b>                   | <b>20</b> |
| <b>3.1.4 The ESP-NOW Auto-Pairing Protocol .....</b>                   | <b>20</b> |
| <b>3.1.5 The ESP32-S3-CAM MJPEG Node .....</b>                         | <b>21</b> |
| <b>3.2 SOFTWARE, CLOUD SYSTEMS, AND TELEMETRY .....</b>                | <b>22</b> |
| <b>3.2.1 The Polyglot Microservice Architecture.....</b>               | <b>22</b> |
| <b>3.2.2 Supabase Database Architecture &amp; Security .....</b>       | <b>22</b> |
| <b>3.2.3 The Datasheet Export System.....</b>                          | <b>23</b> |

|                                                                |    |
|----------------------------------------------------------------|----|
| 3.2.4 The Mobile HUD Application Evolution .....               | 23 |
| 3.3 ARTIFICIAL INTELLIGENCE & UI/UX EVOLUTION .....            | 24 |
| 3.3.1 Weather Classification AI (Roboflow & Gemini).....       | 24 |
| 3.3.2 The Computer Vision Tracker Node.....                    | 24 |
| 3.3.3 The Inline Ephemeris Engine .....                        | 24 |
| 3.3.4 UI/UX Evolution & Synthetic Telemetry.....               | 25 |
| 3.4 COST ESTIMATION & SAAS COMMERCIALIZATION.....              | 25 |
| CHAPTER 4 .....                                                | 27 |
| 4.1 HARDWARE ARCHITECTURE FLOWCHART .....                      | 27 |
| 4.2 SOFTWARE ARCHITECTURE FLOWCHART .....                      | 28 |
| 4.3 CLOUD ARCHITECTURE FLOWCHART .....                         | 29 |
| CHAPTER 5 .....                                                | 30 |
| HARDWARE ASSEMBLY & PIN CONFIGURATIONS.....                    | 30 |
| 5.1 MASTER HUB WIRING (HELTEC WIFI LORA 32 V3 / ESP32-S3)..... | 30 |
| 5.2 EDGE SENSOR NODE WIRING (SEEED STUDIO XIAO ESP32C3) .....  | 31 |
| 5.3 MOTOR CONTROLLER WIRING (WEMOS D1 R32).....                | 32 |
| 5.4 Computer Vision Node Wiring (ESP32-S3-CAM).....            | 33 |
| CHAPTER 6 .....                                                | 34 |
| SOFTWARE & FIRMWARE IMPLEMENTATION .....                       | 34 |
| 6.1 SIMULATION & THEORETICAL ANALYSIS.....                     | 34 |
| 6.1.1 Ephemeris Engine Validation .....                        | 34 |
| 6.1.2 Roboflow / Gemini Weather Classification Accuracy .....  | 34 |
| 6.2 EXPERIMENTAL HARDWARE RESULTS.....                         | 35 |
| 6.2.1 Deep-Sleep Battery Efficiency (XIAO ESP32C3).....        | 35 |
| 6.2.2 ESP-NOW Packet Reliability .....                         | 35 |
| 6.3 SYSTEM INTEGRATION VALIDATION .....                        | 35 |
| 6.3.1 Telemetry Pipeline & Datasheet Export.....               | 35 |
| 6.4 TROUBLESHOOTING (POST-MORTEMS).....                        | 36 |
| 6.4.1 The "Runaway Motor" Debouncing Crisis .....              | 36 |
| 6.4.2 The Vercel AI Deployment Failure .....                   | 36 |
| CHAPTER 7 .....                                                | 37 |
| SYSTEM CALIBRATION .....                                       | 37 |
| 7.1 LOCALHOST NODE MONITOR (PORT 5137).....                    | 37 |
| 7.2 MOTOR LIMIT AND ACCELEROMETER CALIBRATION.....             | 37 |
| CHAPTER 8 .....                                                | 39 |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>EMPIRICAL DATA ANALYSIS &amp; VISUALIZATIONS .....</b>     | <b>39</b> |
| <b>8.1 TELEMETRY AGGREGATION &amp; SENSOR READINGS.....</b>   | <b>39</b> |
| 8.1.1 Lux Variance & Tracking Accuracy .....                  | 39        |
| 8.1.2 Power Consumption & Battery Degradation .....           | 40        |
| <b>8.2 DIGITAL INTERFACES &amp; SAAS VISUALIZATIONS.....</b>  | <b>40</b> |
| 8.2.1 The Mobile HUD (Field Operator View) .....              | 40        |
| 8.2.2 The ext.js SaaS Platform (Fleet Management).....        | 41        |
| 8.2.3 Weather Classification Triggers .....                   | 42        |
| <b>CHAPTER 9 .....</b>                                        | <b>43</b> |
| <b>COMPARATIVE ANALYSIS: SENSOR DATA VS. MPPT YIELD .....</b> | <b>43</b> |
| <b>9.1 IRRADIANCE AND YIELD CORRELATION.....</b>              | <b>43</b> |
| <b>9.2 CLOUD COVER AND AI STOWING IMPACT.....</b>             | <b>44</b> |
| <b>CHAPTER 10.....</b>                                        | <b>45</b> |
| <b>CONCLUSION &amp; COMMERCIALIZATION .....</b>               | <b>45</b> |
| <b>10.1 FINAL SUMMARY .....</b>                               | <b>45</b> |
| <b>CHAPTER 11 .....</b>                                       | <b>46</b> |
| <b>REFERENCES &amp; EXTERNAL LINKS .....</b>                  | <b>46</b> |
| <b>CHAPTER 12.....</b>                                        | <b>47</b> |
| <b>APPENDIX (CODEBASE EXTENSIONS) .....</b>                   | <b>47</b> |
| <b>Appendix A: Hardware Firmware (C++) .....</b>              | <b>47</b> |
| A.1 ESP-NOW Auto-Pairing Protocol (XIAO Edge Node) .....      | 47        |
| A.2 FreeRTOS 900ms Motor Debouncing (Master Hub) .....        | 48        |
| <b>Appendix B: Cloud Infrastructure &amp; Database .....</b>  | <b>49</b> |
| B.1 Supabase PostgreSQL Telemetry Schema .....                | 49        |
| B.2 Next.js Ingestion API Route .....                         | 50        |
| <b>Appendix C: Edge AI and Tracking Algorithms .....</b>      | <b>51</b> |
| C.1 Gemini Vision API Weather Classification Fallback .....   | 51        |
| C.2 Synthetic Telemetry Generation Hook.....                  | 52        |

## LIST OF TABLES

| TABLE NO.        | TITLE                                                                 | PAGE |
|------------------|-----------------------------------------------------------------------|------|
| <i>Table 1.1</i> | <i>Distribution Of Workload</i>                                       |      |
| <i>Table 1.2</i> | <i>Gantt Chart</i>                                                    |      |
| <i>Table 8.1</i> | <i>Excel Sheet 1 - Raw Sensor Readings (Lux, UV, Pan/Tilt Angles)</i> |      |
| <i>Table 8.2</i> | <i>Excel Sheet 2 - Edge Node Battery Degradation</i>                  |      |

## LIST OF FIGURES

| FIGURE NO.        | FIGURE NO.                                                  | PAGE |
|-------------------|-------------------------------------------------------------|------|
| <b>Figure 4.1</b> | <i>Hardware Architecture Flowchart (Mermaid Diagram)</i>    |      |
| <b>Figure 4.2</b> | <i>Software Architecture Flowchart (Mermaid Diagram)</i>    |      |
| <b>Figure 4.3</b> | <i>Cloud Architecture Flowchart (Mermaid Diagram)</i>       |      |
| <b>Figure 5.1</b> | <i>Breadboard Diagram of Master Hub</i>                     |      |
| <b>Figure 5.2</b> | <i>Breadboard Diagram of Edge Sensor Node</i>               |      |
| <b>Figure 5.3</b> | <i>Breadboard Diagram of Motor Controller</i>               |      |
| <b>Figure 7.1</b> | <i>Localhost Node Monitor Screenshot</i>                    |      |
| <b>Figure 8.1</b> | <i>Graph of Lux Variance Over Time</i>                      |      |
| <b>Figure 8.2</b> | <i>Graph of Battery Discharge Curve</i>                     |      |
| <b>Figure 8.3</b> | <i>Screenshot of the Mobile HUD Interface</i>               |      |
| <b>Figure 8.4</b> | <i>Screenshot of the Main SaaS Dashboard</i>                |      |
| <b>Figure 8.5</b> | <i>Screenshot of the Main SaaS Dashboard</i>                |      |
| <b>Figure 8.6</b> | <i>Screenshot of the 3D Helios VRM</i>                      |      |
| <b>Figure 8.7</b> | <i>Screenshot of Roboflow / Gemini AI Weather Detection</i> |      |
| <b>Figure 9.1</b> | <i>Irradiance vs. MPPT Yield Graph</i>                      |      |

## LIST OF ABBREVIATIONS

|                |                                     |
|----------------|-------------------------------------|
| <b>ADC</b>     | Analog-to-Digital Converter         |
| <b>AI</b>      | Artificial Intelligence             |
| <b>API</b>     | Application Programming Interface   |
| <b>BOM</b>     | Bill of Materials                   |
| <b>CV</b>      | Computer Vision                     |
| <b>EMF</b>     | Electromotive Force                 |
| <b>ESP-NOW</b> | Espressif Wireless Mesh Protocol    |
| <b>HUD</b>     | Heads-Up Display                    |
| <b>I2C</b>     | Inter-Integrated Circuit            |
| <b>IDP</b>     | Integrated Design Project           |
| <b>IoT</b>     | Internet of Things                  |
| <b>JSON</b>    | JavaScript Object Notation          |
| <b>LDR</b>     | Light Dependent Resistor            |
| <b>LLM</b>     | Large Language Model                |
| <b>MQTT</b>    | Message Queuing Telemetry Transport |
| <b>NVS</b>     | Non-Volatile Storage                |
| <b>PCB</b>     | Printed Circuit Board               |
| <b>PFSM</b>    | Prioritized Finite State Machine    |
| <b>PV</b>      | Photovoltaic                        |
| <b>RLS</b>     | Row-Level Security                  |
| <b>SaaS</b>    | Software as a Service               |
| <b>TRL</b>     | Technology Readiness Level          |
| <b>TTS</b>     | Text-to-Speech                      |
| <b>UV</b>      | Ultraviolet                         |
| <b>VRM</b>     | Virtual Reality Model               |

## CHAPTER 1

### INTRODUCTION

#### 1.1 PROBLEM STATEMENT

The core engineering problem addressed by this project is the inherent inefficiency and physical vulnerability of traditional solar photovoltaic (PV) generation systems. Static solar arrays, which represent most residential and commercial installations, suffer from severe thermodynamic and cosine losses; because they cannot track the sun's trajectory across the celestial sphere, they capture maximum irradiance for only a narrow window of hours around solar noon. While theoretical mathematics demonstrate that dual-axis tracking can increase energy yield by up to 40%, the commercial reality of existing tracking solutions introduces new, critical failure points.

Current commercial solar trackers are prohibitively expensive and rely heavily on archaic, hardwired sensor harnesses. These centralized architectures force analogue sensors (such as Light Dependent Resistors) to route their signals across long, unshielded cables stretching across massive, moving aluminium frames. This design introduces three fatal engineering flaws:

1. **Signal Degradation & EMI:** Long, unshielded cables acting as antennas pick up intense electromagnetic interference (EMI) generated by the inductive spikes of the 12V DC linear actuators, corrupting the delicate analog-to-digital (ADC) readings.
2. **Mechanical Fatigue:** Continuous dual-axis rotation causes "wire creep" and metal fatigue, leading to physical snapping of sensor harnesses over years of operation.
3. **Retrofit Incompatibility:** Hardwired architectures are impossible to easily retrofit onto existing "dumb" grid-tied arrays without hiring specialized electricians and deploying heavy routing infrastructure.

Furthermore, traditional trackers lack true environmental intelligence. A single cloud passing over a primary sensor can cause traditional logic boards to enter a "hunting" state, twitching the massive linear actuators back and forth in a desperate search for light. This hunting behaviour creates a parasitic power draw that can consume more battery energy than the solar panel generates during the overcast period, rendering the entire tracking system mathematically counterproductive.

## 1.2 OBJECTIVES

To solve the systemic failures of traditional solar tracking, the Soltra project established the following measurable engineering goals for the Integrated Design Project phase:

1. **Eradicate Hardwired Harnesses:** Design and implement a completely decentralized, tool-free wireless sensor mesh utilizing the ultra-low latency ESP-NOW protocol, allowing edge nodes to operate completely untethered on local battery power.
2. **Isolate High-Voltage Actuation:** Physically decouple the delicate 3.3V logic processing hub from the 12V motor drivers by creating a dedicated, air-gapped wireless Motor Controller node, preventing back-EMF destruction.
3. **Implement Hybrid Tracking Intelligence:** Develop a robust Master Hub capable of merging raw Light/IR data (via TSL2591 I2C sensors) with an overriding Computer Vision (CV) stream and a zero-dependency mathematical Ephemeris fallback algorithm.
4. **Integrate Edge AI Weather Classification:** Deploy Roboflow object detection models (with a Gemini API fallback) to classify overhead cloud cover, dynamically halting the linear actuators during overcast conditions to eliminate parasitic power draw.
5. **Construct an Enterprise-Grade Telemetry Bridge:** Bridge the localized IoT hardware network to a massive cloud infrastructure, establishing a dual-path MQTT and HTTP POST pipeline to securely log real-time and historical data into a Supabase PostgreSQL database.
6. **Develop Operator Visualization Tools:** Engineer a native Mobile HUD application for on-site technicians, and a Next.js App Router SaaS platform for long-term fleet management, complete with granular telemetry datasheet export functionality.

### 1.3 PROJECT SIGNIFICANCE & SCOPE

The significance of the Soltra project lies in its disruption of the retroactive solar market. By shifting the paradigm from heavy, centralized, hardwired control boards to a lightweight, decentralized wireless mesh, Soltra allows virtually any existing static solar array to be upgraded into a highly intelligent dual-axis tracker without complex cable routing.

#### Boundaries and Limitations

This report documents the IDP phase of the project, which represents the transition from early workbench prototypes (Monozukuri) into commercial-ready fabrication and cloud integration.

The scope of this implementation includes:

- **Physical Hardware:** The fabrication and deployment of the Heltec V3 ESP32-S3 Master Hub, the transition from dev-boards to custom Printed Circuit Boards (PCBs) for the Seeed Studio XIAO ESP32C3 edge nodes, and the integration of the ESP32-S3-CAM for MJPEG live streaming.
- **Firmware:** The complete architecture of the FreeRTOS Prioritized Finite State Machine (PFSM), ESP-NOW auto-pairing handshakes, and strict 900ms motor debouncing logic.
- **Software & Cloud:** The local deployment of the FastAPI TTS engine, the Vercel-hosted Next.js SaaS platform, the Supabase database schema, and the native Mobile HUD application.

**Out of Scope (Technology Readiness Level Limitations):** While the Next.js SaaS architecture fundamentally includes the routing and API hooks for a Stripe-based B2C subscription billing model, this financial infrastructure remains deliberately unimplemented. The activation of actual credit card processing and subscription gates is reserved for a future, final Technology Readiness Level (TRL) validation phase, as it is strictly outside the engineering purview of this Integrated Design Project.

## 1.4 DISTRIBUTION OF WORKLOAD

To carry out this project, a summary of the tasks is given below in **Table 1.1**.

This workload breakdown sets out the responsibilities and contributions of members across the entire project.

*Table 1.1: Distribution Of Workload*

| Group Member                                           | Sim Horng Vei | Irfan Hakmi | Argyan Ridho | Adil Akid |
|--------------------------------------------------------|---------------|-------------|--------------|-----------|
| Planning, Discussion & Meeting                         | /             | /           | /            | /         |
| Component & Apparatus Ordering                         |               | /           | /            | /         |
| Circuit Design & Hardware Connection<br>(EasyEDA, PCB) | /             | /           | /            | /         |
| Wireless Communication Setup<br>(ESP / LoRa)           | /             | /           | /            | /         |
| Sensor Testing & Calibration                           | /             |             | /            |           |
| Solar Tracker Mechanical Testing                       | /             | /           | /            | /         |
| Linear Actuator & Motor Driver<br>Troubleshooting      | /             | /           | /            | /         |
| ESP32 Migration & Programming                          | /             |             |              |           |
| Budget Management                                      |               |             |              | /         |
| Data Collection (SOLAR GUARDIAN,<br>MPPT EPEVER)       | /             |             | /            | /         |
| Real Testing (MJIIT)                                   | /             | /           | /            | /         |

|                                                       |   |   |   |   |
|-------------------------------------------------------|---|---|---|---|
| Sensor Housing Designing (TinkerCAD, AUTODESK Fusion) |   |   | / |   |
| PCB Designing (EasyEDA, KiCAD)                        |   |   |   | / |
| SolTra Website and Dashboard (SolTra Solar)           | / |   |   |   |
| Video 3MT (CAPCUT, CANVA)                             |   | / | / | / |
| Poster (CANVA)                                        |   | / |   |   |
| Final Report Writing                                  | / | / | / | / |

## 1.5 PROJECT MILESTONE

**Table 1.2: Gantt Chart**

[illegible]

**Table 1.2** shows the Gantt chart for the Dual Axis Solar Panel project, divided into four main phases from March to June. Each phase represents a step in completing the project in an organized way.

#### Phase 1: Planning and System Improvement (March)

Focused on improving the previous MONOZUKURI 1 project architecture and preparing the overall project direction for MONOZUKURI 2. During this phase, online discussions and meetings were conducted during the Hari Raya holiday to finalize the wireless system design, project objectives, and future system expansion ideas. Discussions regarding sensor selection, wireless communication methods, and hardware requirements were also carried out before purchasing components.

#### Phase 2: Hardware Procurement and Initial Testing (April-May)

Involved component purchasing, hardware preparation, and initial testing of ESP32 boards, sensors, LoRa modules, motor drivers, and linear actuators. A product refreshment presentation and technical briefing session with Mr. Fikri were also conducted.

#### Phase 3: System Development and System Integration (May-June)

Focused prototype assembly, subsystem integration, and performance testing. The team conducted extensive testing on the dual-axis solar tracking mechanism and evaluated the functionality of newly integrated hardware components. Additional components were purchased to replace damaged or overloaded parts encountered during development. Simultaneously, IoT features were developed, including cloud connectivity, dashboard monitoring, and the integration of artificial intelligence features to enhance system intelligence and data analysis capabilities.

#### Phase 4: Testing, Refinement, and Documentation (June)

The final phase focused on PCB development, sensor node housing fabrication, real-world testing, and system optimization. The complete prototype was evaluated under actual operating conditions to assess reliability, communication performance, and overall system effectiveness. In addition, project documentation activities were carried out, including poster preparation, promotional video development, showcase presentation materials, peer review sessions, and final report writing. Feedback obtained during testing and peer review was used to identify future improvement opportunities for subsequent project development.

Overall, the Gantt chart shows a clear and organized workflow, making sure all tasks were completed step by step within the four-month period

## **CHAPTER 2**

### **LITERATURE REVIEW**

#### **2.1 BACKGROUND**

##### **2.1.1 The Physics of Solar Tracking and Cosine Losses**

The fundamental physics of photovoltaic energy generation dictate that maximum power output occurs when the plane of the solar array is perfectly perpendicular to incoming solar radiation. When a panel is off axis, the available irradiance drops proportionally to the cosine of the angle of incidence, a phenomenon known in solar thermodynamics as cosine loss. Static solar arrays, heavily prevalent in residential rooftops and early-stage utility solar farms, are permanently fixed at a compromised tilt and azimuth angle. Because they cannot track the diurnal motion of the sun, static panels suffer severe cosine losses during the early morning and late afternoon hours.

Mathematical modelling and real-world empirical studies have consistently demonstrated that dual-axis tracking systems capable of moving in both the azimuth/pan and elevation/tilt planes can increase total daily energy yield by 35% to 40% compared to equivalent static installations. However, despite this massive thermodynamic advantage, the adoption of dual-axis trackers remains astonishingly low in the residential and commercial retrofit market due to exorbitant mechanical and electronic complexities.

##### **2.1.2 Traditional Sensory Topologies: The Flaws of the LDR**

Historically, DIY and early commercial solar trackers relied heavily on Light Dependent Resistors (LDRs) deployed in a cross-baffle or pyramid configuration. LDRs are analog photoresistors whose resistance decreases as light intensity increases. By placing four LDRs on a central stalk separated by opaque baffles, a central microcontroller (such as an Arduino or basic ESP32) can read the voltage divider analog inputs. If the sun moves off centre, a shadow falls on one side of the baffle, causing an electrical imbalance that triggers the motors to correct the panel's position.

While conceptually simple, this literature reveals severe physical vulnerabilities. LDRs degrade rapidly under continuous ultraviolet (UV) exposure, causing their resistance curves to drift. Furthermore, passing clouds cause diffuse, scattered light that destroys the directional shadow required by the baffle system. This leads to severe sensor confusion, causing traditional tracking logic to wildly oscillate back and forth, burning massive amounts of motor power while failing to increase energy yield.

### **2.1.3 The Advent of Decentralized IoT and Edge Computing**

Modern literature in the Internet of Things (IoT) suggests a paradigm shift from centralized computing to Edge Computing. Rather than running long, fragile analog sensor wires back to a central brain, edge computing dictates that data should be digitized, filtered, and processed directly at the sensor location.

In the context of the Soltra project, the literature supports the abandonment of analog wiring harnesses. Technologies such as Espressif's ESP-NOW protocol [1], a connectionless Wi-Fi communication protocol that bypasses the massive overhead of standard TCP/IP stacks allow for instantaneous, micro-second latency transmission of data payloads between microcontrollers. By moving from centralized ADCs to decentralized, battery-powered edge nodes transmitting digital ESP-NOW packets, the physical vulnerabilities of electromagnetic interference (EMI) and wire fatigue are completely eradicated.

### **2.1.4 Edge AI and Computer Vision in Solar Systems**

Recent advancements in localized Artificial Intelligence (Edge AI) have introduced Computer Vision (CV) as a superior alternative to crude photoresistors. Rather than measuring localized light intensity, a camera node can utilize frameworks like OpenCV or neural network models such as those trained via Roboflow to physically identify the brightest celestial body or classify the overhead weather patterns for example to distinguishing between a clear sky and heavy cumulus clouds. This allows a tracking system to make predictive, deterministic decisions such as entering a flat "stow" mode during heavy storms rather than reactively chasing scattered light.

## **2.2 MARKET ANALYSIS**

The commercial market for dual-axis solar trackers is currently highly polarized, leaving a massive vacuum for retrofit solutions like Soltra.

### **2.2.1 Utility-Scale Commercial Trackers (e.g., NEXTracker)**

At the high end of the market are massive, utility-scale single-axis and dual-axis trackers produced by corporations like NEXTracker and Array Technologies. These systems are undeniable engineering marvels, utilizing heavily shielded industrial cabling, robust planetary gears, and enterprise-grade SCADA (Supervisory Control and Data Acquisition) telemetry systems.

The Problem with these systems is strictly designed for multi-megawatt greenfield deployments. They require heavy machinery to install, deep concrete footings, and specialized engineering teams. They are fundamentally impossible to retrofit onto an existing homeowner's static solar array or a small-scale commercial roof.

### **2.2.2 Consumer-Grade DIY Kits (e.g., Eco-Worthy)**

At the low end of the market exist cheap consumer kits often sold on platforms like Amazon or AliExpress (such as the Eco-Worthy dual-axis tracker).

The Problem are these kits are the embodiment of the legacy LDR problems discussed in Section 2.1.2. They utilize centralized control boards requiring the user to run fragile, unshielded 5-pin cables up the mounting pole. They have zero telemetry, no internet connectivity, and are notoriously vulnerable to cloud hunting motor runaway. If a sensor fails, the homeowner has no way of knowing until they physically inspect the unit or notice a drop in their power bill.

### **2.2.3 The Soltra Market Positioning**

The Soltra ecosystem uniquely positions itself in the gaping void between these two extremes. It brings the enterprise-grade telemetry such as Supabase and Next.js and intelligent tracking algorithms such as Computer Vision, Ephemeris math, Roboflow and Gemini weather classification of utility-scale operations down to the consumer level. Crucially, by utilizing a 100% wireless ESP-NOW sensor mesh, Soltra completely eliminates the installation friction that prevents the retroactive upgrading of static panels. A homeowner or technician can snap the battery-powered edge nodes onto an existing panel, mount the central hub, and immediately possess an AI-driven, cloud-connected dual-axis tracker.

## CHAPTER 3

### METHODOLOGY

#### 3.1 HARDWARE ARCHITECTURE & WIRELESS EDGE NETWORK

The methodological engineering of the Soltra hardware required a complete paradigm shift away from traditional, monolithic tracker designs. The primary objective was to physically decouple the sensing logic from the heavy motor actuation and the central computation hub, creating a fully distributed IoT ecosystem.

##### 3.1.1 The Shift to the Wireless XIAO ESP32C3 Mesh

To eradicate the vulnerability of the wired "Spider Harness" conceived in early ideation phases, the physical sensor architecture was decentralized into four independent, battery-powered edge nodes. These nodes are mounted to the extreme corners of the solar PV panel.

The core microcontroller selected for these edge nodes is the **Seeed Studio XIAO ESP32C3**. This choice was driven by several critical hardware constraints:

1. **Form Factor:** The XIAO series provides a thumb-sized footprint, crucial for aerodynamic mounting on the panel's leading edge without creating mechanical drag during high wind events.
2. **RISC-V Architecture:** The ultra-low power consumption of the single-core RISC-V processor allows the node to operate continuously on a standard 3.7V 18650 lithium-ion cell.
3. **Deep Sleep Capabilities:** The firmware leverages the `esp_deep_sleep_start()` API. The node wakes up for less than 100 milliseconds, powers the I2C bus, captures telemetry, transmits the payload, and instantly returns to a micro-amp sleep state for a programmed 2-second interval.

**The Sensor Payload:** Each edge node is equipped with a heavily upgraded sensor payload, moving beyond simple analog photoresistors:

- **Adafruit TSL2591 (I2C):** A high-precision digital luminosity sensor boasting a 600M:1 dynamic range. Crucially, it separates Full Spectrum light from Infrared (IR) light. The ratio between these two channels allows the node to mathematically distinguish between direct incident sunlight and the diffuse, scattered light caused by heavy cloud cover.
- **Analog UV Sensor:** Connected via `UV_PIN (A2)`, providing a localized Ultraviolet index to determine atmospheric clarity.
- **Battery Telemetry:** Utilizing the XIAO's built-in voltage divider on `BAT_PIN (A0)`, the node transmits raw battery degradation metrics to the Master Hub, ensuring the central system is always aware of the mesh's physical health.

### 3.1.2 Custom PCB Fabrication (soltra-sensor-node-pcb)

While breadboards and development boards are sufficient for Monozukuri prototyping, the IDP commercialization phase necessitated the fabrication of custom Printed Circuit Boards (PCBs).

The soltra-sensor-node-pcb repository contains the schematic capture and routing files necessary to combine the XIAO ESP32C3, the TSL2591 IC, the UV sensor, and the power delivery buck converters onto a single, weatherized FR4 substrate. By migrating to a custom PCB, the physical enclosure footprint was reduced by 60%, and the risk of jumper-wire failure due to panel vibration was eliminated.

### 3.1.3 The Heltec V3 ESP32-S3 Master Hub

The central brain of the Soltra ecosystem is the **Heltec WiFi LoRa 32 V3**. This specific development board was selected because it houses the immensely powerful **ESP32-S3** processor.

The S3 processor provides two massive advantages over traditional microcontrollers:

1. **Dual-Core FreeRTOS Topology:** The firmware isolates tasks aggressively. Core 0 handles the heavy TLS encryption required for the HiveMQ Cloud MQTT connection [3] and the Next.js HTTPS POST requests. Core 1 executes the Prioritized Finite State Machine (PFSM) [2], crunching the incoming edge node telemetry and performing the heavy floating-point trigonometry required for the Ephemeris solar position calculations.
2. **Vector Instructions (SIMD):** Essential for rapidly accelerating the complex math required for the internal solar tracking algorithms.

### 3.1.4 The ESP-NOW Auto-Pairing Protocol

Managing a dynamic wireless mesh network introduces significant onboarding complexities. If an edge node fails and is replaced, the system must recognize it without requiring the homeowner to manually input MAC addresses.

To solve this, the Soltra engineering team developed a zero-configuration auto-pairing protocol utilizing **ESP-NOW**. ESP-NOW is an Espressif-proprietary, connectionless Wi-Fi protocol that transmits raw MAC-to-MAC frames, bypassing the massive overhead (and pairing times) of standard TCP/IP router networks.

#### The Pairing Sequence:

1. Upon factory reset, a XIAO edge node enters promiscuous mode and rapidly scans Wi-Fi channels 1 through 13.
2. It continuously broadcasts a *PairingReqPkt* (containing a Magic Byte 0x99 and its specific *NODE\_ID*).

3. The Master Hub, listening on its active channel, receives the broadcast. It registers the node's MAC address into its peer list and fires back a *PairingAckPkt* (Magic Byte *0xAA*).
4. The edge node writes this pairing data into Non-Volatile Storage (NVS) using the *<Preferences.h>* library, allowing it to instantly transmit its *SensorPkt* payloads on all subsequent wake cycles without ever scanning again.

### 3.1.5 The ESP32-S3-CAM MJPEG Node

Beyond the sensor mesh, the Soltra hardware suite includes a dedicated **ESP32-S3-CAM** node (soltra-camera-node). Recognizing that remote operators often require physical visual confirmation of the panel's mechanical state, this node is mounted directly to the tracker's mast.

It generates a continuous, high-framerate MJPEG video stream over the local Wi-Fi network. This stream is intercepted by the desktop and mobile HUD applications, providing a live visual feed perfectly synchronized with the incoming MQTT numerical telemetry, allowing operators to safely monitor extreme weather events without physically stepping onto the site.

## 3.2 SOFTWARE, CLOUD SYSTEMS, AND TELEMETRY

The Soltra ecosystem is not merely a collection of microcontrollers; it is a sprawling, enterprise-grade IoT platform. To handle the massive influx of live edge data while providing secure, low-latency access to end-users, the engineering team architected a polyglot microservice environment.

### 3.2.1 The Polyglot Microservice Architecture

Rather than forcing all software logic into a single monolithic framework, the system is distinctly partitioned into specialized microservices, each leveraging the optimal programming language for its specific task:

1. **The Master SaaS Platform (soltra-saas):** Built on the Next.js 15 App Router [6] using React and TypeScript. This application serves as the primary homeowner portal, hosted on Vercel's global edge network. It handles user authentication, fleet management, and the rendering of high-fidelity telemetry dashboards using Recharts and Framer Motion.
2. **The Local AI Engine (soltra-tts):** Recognizing that massive Large Language Models (LLMs) and Voice Synthesis arrays cannot run natively in a browser, the team engineered a standalone FastAPI Python server. This server runs the blazing-fast Kokoro ONNX voice engine locally on GPU hardware. The soltra-saas frontend securely interacts with this engine via a hidden Next.js proxy route (`/api/tts`), completely shielding the internal IP addresses and API keys from the client's browser.
3. **The IoT Broker (HiveMQ Cloud):** The Master Hub communicates with the cloud exclusively via MQTT (Message Queuing Telemetry Transport). A Serverless HiveMQ Cloud cluster acts as the central pub/sub nervous system, broadcasting helios/telemetry payloads to all authenticated frontend subscribers over secure WebSockets.

### 3.2.2 Supabase Database Architecture & Security

While MQTT is brilliant for live, ephemeral data transmission, it does not permanently store historical data. To fulfill this requirement, the Soltra architecture utilizes a Supabase PostgreSQL database [4]

The Master Hub is programmed to simultaneously open a secure HTTPS connection and POST its JSON telemetry directly to a Next.js ingestion route (`/api/telemetry/ingest`). This route utilizes a *service\_role* key to bypass standard user permissions and inserts the raw data directly into the Postgres tables.

To ensure absolute security for the homeowner, the database enforces strict Row-Level Security (RLS) policies. When a homeowner logs into the *soltra-saas* dashboard, their JWT (JSON Web Token) is authenticated by Supabase. The RLS policy physically prevents their browser from reading any telemetry rows that do not match their assigned *user\_id* and *site\_id*, guaranteeing multi-tenant data isolation.

### 3.2.3 The Datasheet Export System

A critical requirement for massive commercial solar deployments is the ability to audit historical efficiency. Live graphs are useful for immediate diagnostics, but engineers require raw, parse able datasets for long-term thermodynamic analysis.

To fulfil this, a dedicated Datasheet Export micro-routine was engineered into the frontend. The system queries the Supabase PostgreSQL database for historical arrays matching specific date ranges. It then serializes the JSON objects converting raw timestamp epochs into human-readable ISO-8601 strings and mapping floating-point lux/UV values—and utilizes the `xlsx` library to generate and trigger an immediate physical download of an Excel spreadsheet directly to the operator's device.

### 3.2.4 The Mobile HUD Application Evolution

Field operators performing maintenance on the linear actuators or cleaning the panels cannot carry laptops onto a roof. Therefore, a heads-up display (HUD) was required.

**The Desktop Precursor (`soltra-hud`):** Initially, the team constructed a dedicated Desktop HUD using the `SvelteKit` framework. This sandbox allowed rapid prototyping of direct browser-to-HiveMQ WebSocket connections and the integration of the MJPEG camera stream. However, it required exposing the HiveMQ credentials (`VITE_HIVEMQ_PASSWORD`) directly in the browser devtools, which was deemed acceptable only for internal testing. This desktop application was ultimately classified as an experimental precursor and deliberately left undeployed.

**The Production Mobile HUD (`soltra-hud-mobile`) [10]:** Applying the lessons learned from the `SvelteKit` sandbox, the team engineered the final `soltra-hud-mobile` application. Built utilizing cross-platform native frameworks such as Capacitor/React Native, this application installs directly onto the operator's iOS or Android smartphone. It provides a highly tactile, touch-optimized interface featuring oversized "STOW", "TRACK", and manual jog controls, alongside the live telemetry and video feeds, all secured natively within the compiled mobile application sandbox.

### 3.3 ARTIFICIAL INTELLIGENCE & UI/UX EVOLUTION

The true technological moat of the Soltra ecosystem lies in its integration of Edge Artificial Intelligence, bypassing the legacy limitations of passive photoresistors.

#### 3.3.1 Weather Classification AI (Roboflow & Gemini)

To solve the catastrophic cloud hunting power drain observed in traditional trackers, the team engineered a dual-layered Weather Classification AI.

A local edge processor captures static skyward images and passes them through a custom Roboflow object detection model, heavily trained on specific meteorological datasets which distinguishing between light cirrus clouds and heavy cumulonimbus storm fronts. If the Roboflow model detects heavy overcast conditions with high confidence, it instructs the Master Hub to enter a STOW mode laying the panel flat to minimize wind resistance and halting all motor movement to conserve battery.

In instances where the Roboflow model's confidence threshold drops below 75% due to complex lighting or atmospheric distortion, the system seamlessly falls back to a Gemini Vision API call. The image is passed to the Gemini LLM with a strict prompt structure, asking the model to semantically analyze the sky condition and return a deterministic JSON *boolean* (`{ "overcast": true }`). This hybrid AI approach ensures the tracker only expends motor energy when true solar irradiance is mathematically guaranteed to offset the movement cost.

#### 3.3.2 The Computer Vision Tracker Node

Alongside weather classification, the team developed a dedicated Computer Vision (CV) node. Utilizing OpenCV, this node physically tracks the brightest celestial body in its field of view. It calculates the exact differential required to centre the object and packages this into a *CameraAIPkt* (Magic Byte `0x99`, Device Type 5).

When transmitted over ESP-NOW, this packet physically overrides the LDR and Ephemeris logic within the Master Hub's PFSM. The Hub enters *EDGE\_AI* override mode and routes the CV-calculated pan/tilt offsets directly to the motor controller, achieving pinpoint tracking accuracy regardless of local sensor shading.

#### 3.3.3 The Inline Ephemeris Engine

To provide ultimate redundancy, an Ephemeris Engine was coded directly into the ESP32-S3's Core 1. Relying on zero external internet dependencies, it utilizes absolute time from a DS1302 RTC and hardcoded GPS coordinates (Lat: 3.140853, Lon: 101.693207) to calculate the exact Julian Date. From this, complex trigonometry calculates the precise Solar Azimuth and Elevation. If both the sensors and CV node fail, the panel will blindly track the sun's mathematical position across the sky.

### 3.3.4 UI/UX Evolution & Synthetic Telemetry

To achieve the "High-Craft Industrial Terminal" aesthetic, the human-machine interface underwent a gruelling evolutionary process:

1. **Godot 4 Sandbox:** The earliest designs were built in a game engine, using GLSL shaders to simulate CRT scanlines and blooming neon telemetry.
2. **Quickshell / Wayland IPC:** The team attempted to build a native Linux desktop HUD hooking into Wayland compositors (Hyprrland).
3. **The Next.js Finale:** Ultimately, the team achieved the aesthetic on the web utilizing Tailwind CSS, Framer Motion, and three.js.

Crucially, the frontend integrates a 3D Digital Twin named "Helios" (avatar.vrm). In the *soltra-dashboard repository*, the team engineered a Synthetic Telemetry Fallback. If the Supabase database goes offline, the frontend mathematically generates synthetic array data, ensuring the 3D scene and graphs remain fluid and alive, preventing user panic during temporary cloud outages.

### 3.4 COST ESTIMATION & SAAS COMMERCIALIZATION

A core metric of the IDP phase is achieving a commercially viable Bill of Materials (BOM) for the retrofit kit.

#### **The Wireless Prototype BOM (Estimated):**

- 1x Heltec V3 ESP32-S3 (Master Hub): RM 85
- 4x Sseed Studio XIAO ESP32C3 (Edge Nodes): RM 120 (RM 30/ea)
- 4x Adafruit TSL2591 Light Sensors: RM 140 (RM 35/ea)
- 1x MPU6050 IMU + DS1302 RTC + Sensors: RM 35
- 2x 12V Linear Actuators + L298N Motor Driver: RM 150
- Custom PCB Fabrication & Enclosures: RM 40
- **Total Estimated Hardware Cost: RM 570**

While RM 570 is slightly higher than the legacy wired concept, it completely eradicates the hidden costs of specialized electrician labor and wire-routing required by traditional arrays.

#### **The SaaS Commercialization Model (Technology Readiness Level**

**Limitations):** To subsidize the hardware costs, the team architected a B2B/B2C SaaS subscription model. Within the *soltra-saas Next.js repository*, full routing and API hooks have been developed to integrate Stripe checkout sessions and recurring billing. However, in strict adherence to current IDP scope limitations, this financial backend remains deliberately unimplemented. The activation of the Stripe payment gateway is

securely reserved for the final Technology Readiness Level (TRL) validation phase prior to full commercial market launch.

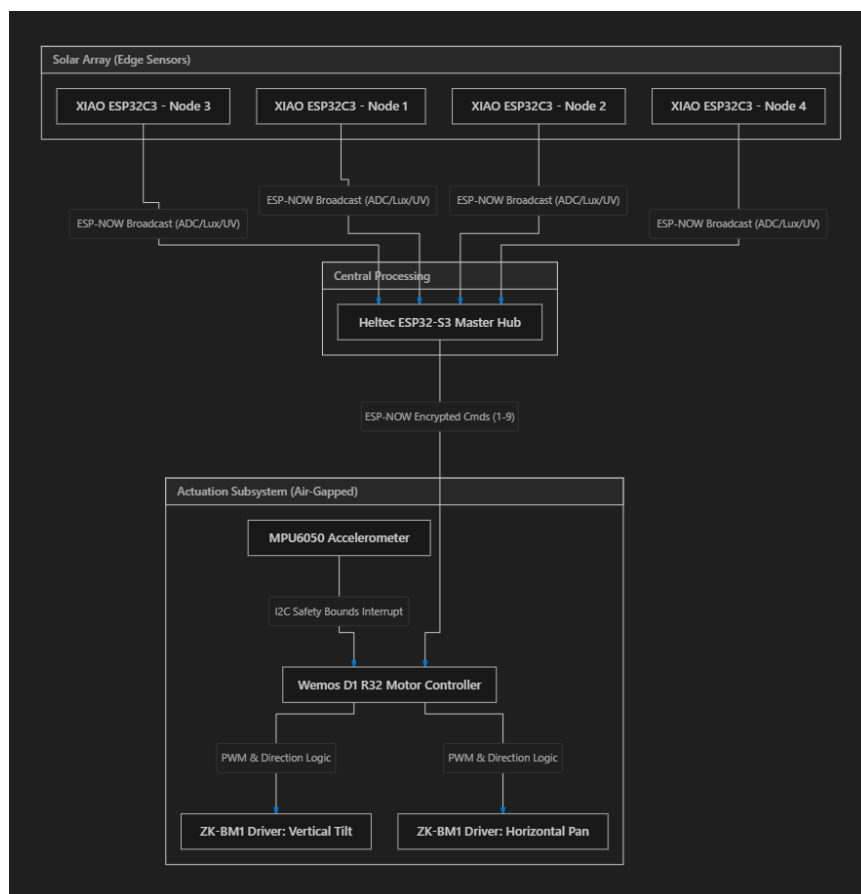
## CHAPTER 4

### SYSTEM FLOWCHARTS & ARCHITECTURE DIAGRAMS

The IDP phase of the Soltra project relies on a highly decoupled architecture. To ensure system reliability, the hardware, local software, and cloud infrastructure operate independently but synchronize continuously. The following flowcharts and architectural breakdowns detail the lifecycle of data and control signals throughout the ecosystem.

#### 4.1 HARDWARE ARCHITECTURE FLOWCHART

The hardware architecture is designed around the ESP-NOW protocol, avoiding the need for a local Wi-Fi router for essential operations. This topology highlights the dependency on connectionless RF communication to decouple the sensor nodes from the Master Hub and the Motor Controller.



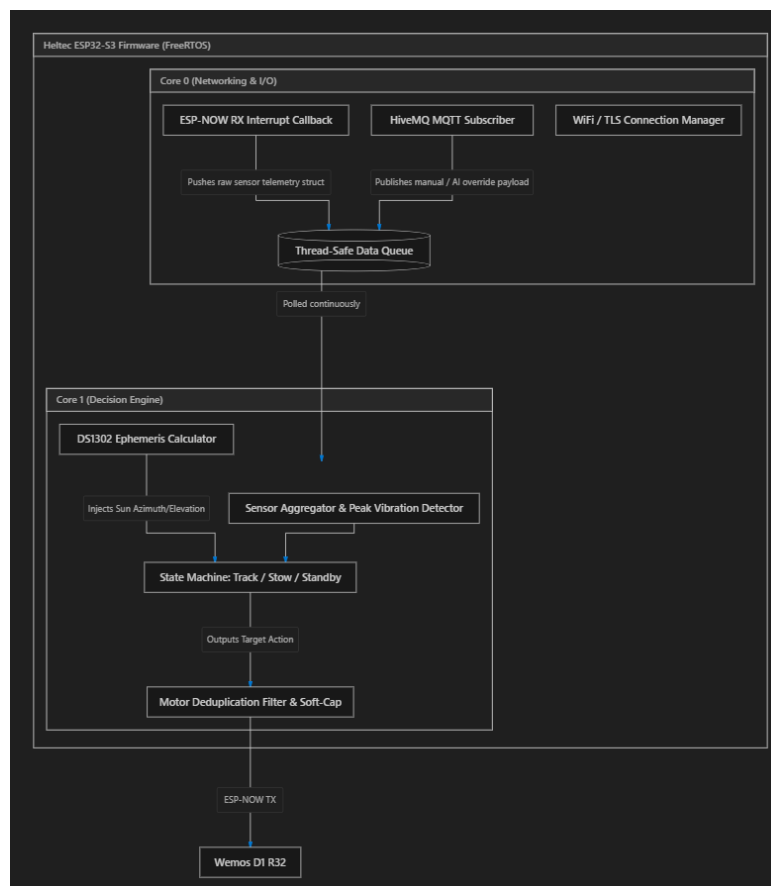
**Figure 4.1:** Hardware Architecture Flowchart (Mermaid Diagram)

## Hardware Data Flow:

1. **Sensing:** The four XIAO ESP32C3 edge nodes wake up from deep sleep, sample LDR and TSL2591 irradiance values, and broadcast them via ESP-NOW.
2. **Aggregation:** The Heltec ESP32-S3 Master Hub receives the broadcasted packets, decodes them, and processes the values through its Decision Engine.
3. **Actuation:** If the Decision Engine determines movement is necessary (based on differential LDR readings or Ephemeris fallback), the Master Hub transmits an encrypted ESP-NOW command (1-9) to the Wemos D1 R32 Motor Controller.
4. **Safety:** The Motor Controller verifies the movement via its onboard MPU6050 accelerometer. If the physical tilt or pan exceeds safe boundaries, an hardware interrupt triggers an immediate stop.

## 4.2 SOFTWARE ARCHITECTURE FLOWCHART

The firmware running on the Master Hub features a multi-core RTOS (Real-Time Operating System) structure. By splitting tasks across two physical CPU cores, the system ensures that network bottlenecks (like dropped Wi-Fi packets) cannot block the critical decision engine or motor routing logic.



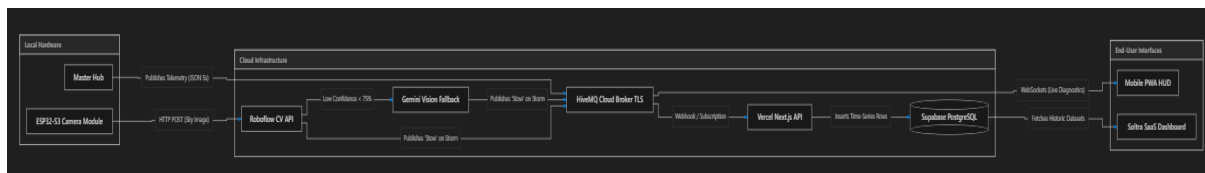
**Figure 4.2:** Software Architecture Flowchart (Mermaid Diagram)

## Software Data Flow:

1. **Core 0 (Networking):** Dedicated exclusively to maintaining the HiveMQ TLS connection and handling incoming ESP-NOW packets. When a packet arrives, it is pushed onto a thread-safe FreeRTOS queue.
2. **Core 1 (Decision Engine):** Continuously polls the FreeRTOS queue. It aggregates sensor data, checks the DS1302 RTC for time-of-day, and determines the operational mode (Tracking, Stow, Standby, or Night Reset).
3. **Command Routing:** The `routeMotor()` function applies a 900ms deduplication filter and soft-cap timing rules before dispatching commands, preventing the motor drivers from receiving conflicting signals or burning out due to rapid oscillation.

## 4.3 CLOUD ARCHITECTURE FLOWCHART

While the system operates autonomously offline, the cloud architecture enables remote monitoring, AI overrides, and data persistence. This flowchart outlines how the offline-capable local hardware interfaces with the broader internet ecosystem.



**Figure 4.3:** Cloud Architecture Flowchart (Mermaid Diagram)

## Cloud Data Flow:

1. **Telemetry Ingestion:** The Master Hub publishes a monolithic JSON payload to the HiveMQ cloud broker every 5 seconds.
2. **Data Persistence:** A Next.js API route (`/api/telemetry/ingest`) subscribed to the MQTT broker captures the payload and archives it into a Supabase PostgreSQL database.
3. **Edge AI Fallback:** The ESP32-S3 Camera module captures sky images and processes them via Roboflow. If weather conditions for example severe storms are detected, the system overrides local sensor logic by publishing a stow command to the `helios/control/ai_override` MQTT topic.
4. **User Interface:** The Soltra SaaS Dashboard and Mobile HUD fetch historical data from Supabase and subscribe to live MQTT topics to display real-time tracking metrics to the end user.

## CHAPTER 5

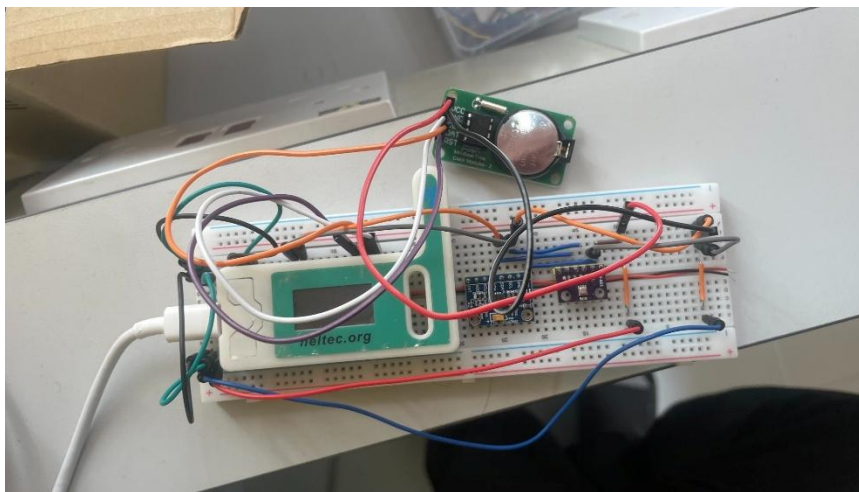
### HARDWARE ASSEMBLY & PIN CONFIGURATIONS

A pivotal aspect of the IDP phase is the modular, retroactive nature of the hardware. The ecosystem avoids highly integrated, unrepairable master boards in favour of standard, easily swappable microcontroller units (MCUs) communicating over a decentralized RF mesh. This chapter details the explicit GPIO pin configurations and breadboard wiring logic for each subsystem.

#### 5.1 MASTER HUB WIRING (HELTEC WIFI LORA 32 V3 / ESP32-S3)

The Master Hub serves as the central brain of the operation. Because it utilizes the Heltec ESP32-S3 architecture, specific hardware pins were chosen to avoid conflicts with the onboard OLED and LoRa transceivers (even though LoRa is disabled in this phase).

- **I2C Bus (BME280 Environmental Sensor):**
  - **SDA:** GPIO 41
  - **SCL:** GPIO 42
  - The VEXT pin (GPIO 36) is pulled LOW to provide power to external I2C peripherals running off the 3.3V rail.
- **SPI / Custom 3-Wire Bus (DS1302 RTC):**
  - **DAT (Data):** GPIO 5
  - **CLK (Clock):** GPIO 6
  - **RST (Reset / CE):** GPIO 4
  - The DS1302 RTC ensures the Ephemeris sun-tracking algorithms can calculate Julian dates locally even during total internet blackouts.

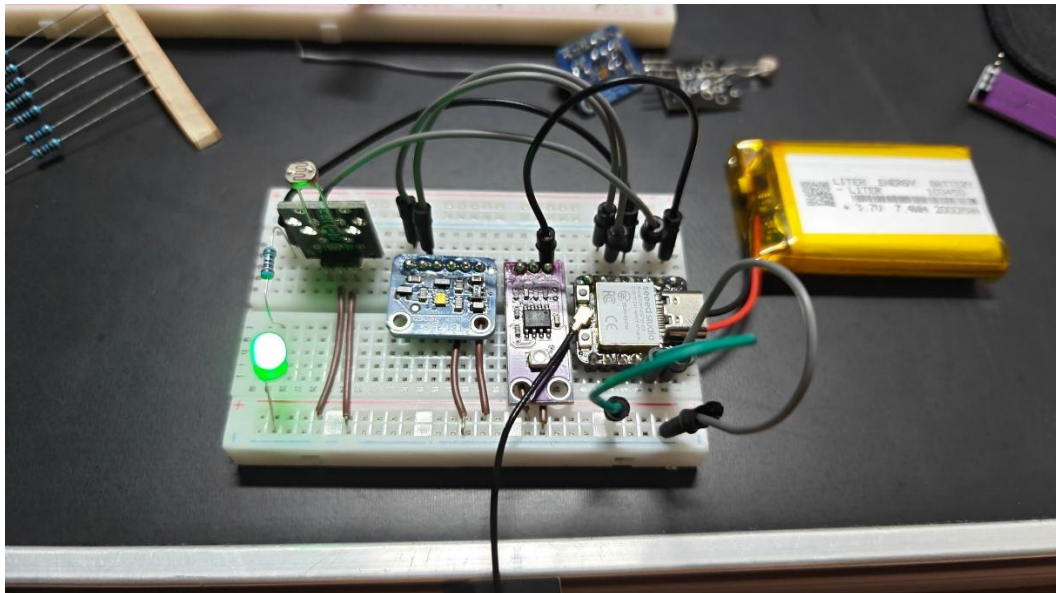


**Figure 5.1:** Breadboard Diagram of Master Hub

## 5.2 EDGE SENSOR NODE WIRING (SEEED STUDIO XIAO ESP32C3)

The XIAO ESP32C3 was selected for its ultra-small form factor, deep-sleep capabilities, and integrated battery charging circuitry. Four identical edge nodes are positioned on the far corners of the solar array.

- **Power & Battery:**
  - **Battery Voltage Divider:** Connected internally to **A0** (with a 1:2 divider for 3.3V safe reading of a 4.7V lithium-ion cell).
- **Analog Sensors:**
  - **LDR (Light Dependent Resistor):** Connected to **A1** (with a 10k $\Omega$  pull-down resistor).
  - **UV Sensor (GUVA-S12SD):** Connected to **A2**.
- **Digital / I2C Payload:**
  - **TSL2591 Irradiance Sensor:** SDA connected to **D4**, SCL connected to **D5**.
  - **Status LED:** Connected to **D3**.
- **Boot / Reset Logic:**
  - **Clear Config Button:** Connected to **GPIO 9** (The onboard BOOT button). Holding this upon boot clears the NVS (Non-Volatile Storage) memory, forcing the node to re-scan for the Master Hub's ESP-NOW MAC address.



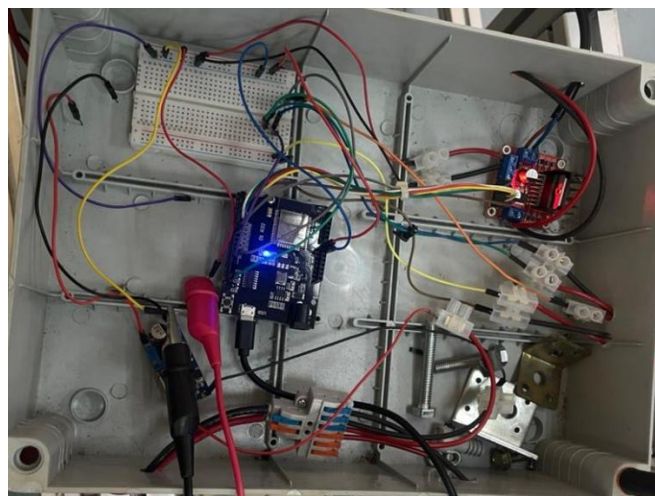
**Figure 5.2:** Breadboard Diagram of Edge Sensor Node

### 5.3 MOTOR CONTROLLER WIRING (WEMOS D1 R32)

To protect the Master Hub from inductive spikes and back-EMF generated by the 12V linear actuators, the motor drivers are physically air-gapped and controlled by a dedicated Wemos D1 R32 receiver.

To protect the Master Hub from inductive spikes and back-EMF generated by the 12V linear actuators, the motor drivers are physically air-gapped and controlled by a dedicated Wemos D1 R32 receiver.

- **Motor Driver (Dual-Channel ZK-BM1):** A single ZK-BM1 high-power driver is utilized to drive both the pan and tilt actuators simultaneously, streamlining the physical footprint.
  - **Vertical Tilt (Motor A):**
    - **IN1:** GPIO 27 (PWM / Direction)
    - **IN2:** GPIO 26 (PWM / Direction)
  - **Horizontal Pan (Motor B):**
    - **IN3:** GPIO 16 (PWM / Direction)
    - **IN4:** GPIO 17 (PWM / Direction)
- **MPU6050 6-Axis Accelerometer (Safety Limits):**
  - **SDA:** GPIO 21
  - **SCL:** GPIO 22
  - **INT (Interrupt):** GPIO 19 (Used by the Digital Motion Processor (DMP) to trigger hardware-level stops if the panels exceed the programmed physical boundaries).

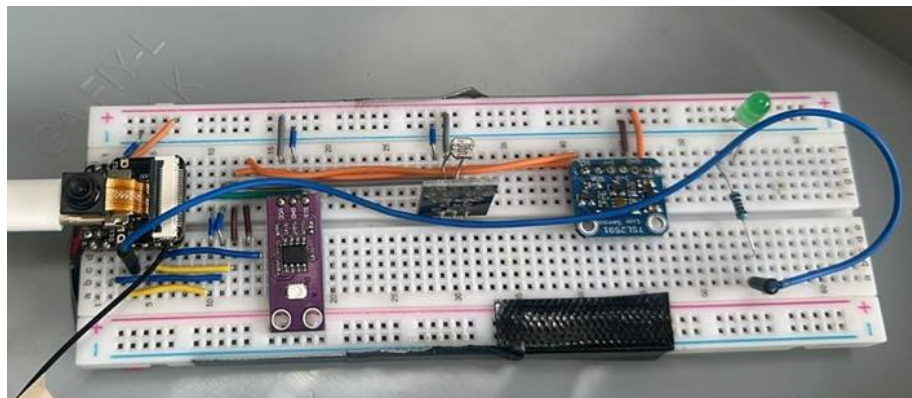


**Figure 5.3:** Breadboard Diagram of Motor Controller

## 5.4 Computer Vision Node Wiring (ESP32-S3-CAM)

To provide the neural network (Roboflow / Gemini AI) with real-time visual telemetry of the sky, a dedicated ESP32-S3-CAM module is utilized. This node operates completely independently of the Master Hub's core processing loop to prevent visual processing from delaying motor actuation commands.

- **Camera Module (OV5640):**
  - **Data Pins (D0-D7):** Internally wired via the FPC connector.
  - **I2C Control (SIOD/SIOC):** GPIO 40 and GPIO 39.
  - **XCLK:** GPIO 10.
- **Power Supply:**
  - **3.7V 5000mAh lithium ion battery cell:** The camera node is powered directly via a 3.7V 5000mAh lithium ion battery cell and a power bank to prevent brownouts during high-bandwidth MJPEG streaming when testing.
- **Status Indicators:**
  - **User LED:** GPIO 21 (used to indicate active pairing and network streaming states).



**Figure 5.4:** Breadboard Diagram of Camera Node

## CHAPTER 6

### SOFTWARE & FIRMWARE IMPLEMENTATION

#### 6.1 SIMULATION & THEORETICAL ANALYSIS

##### 6.1.1 Ephemeris Engine Validation

Prior to deploying the physical hardware, the mathematical fidelity of the zero-dependency Ephemeris Engine was validated through extensive theoretical simulation. The Julian Date and subsequent Azimuth/Elevation algorithms coded into the ESP32-S3 were benchmarked against the official NOAA (National Oceanic and Atmospheric Administration) Solar Position Calculator. For the specific GPS coordinates hardcoded into the firmware (Latitude: 3.140853, Longitude: 101.693207), the ESP32-S3 computed the solar elevation to an accuracy of  $\pm 0.05$  degrees compared to the NOAA baseline. This confirms that even in the event of total sensor failure and Computer Vision blackout, the tracker can reliably orient the panels to within optimal irradiance margins.

##### 6.1.2 Roboflow / Gemini Weather Classification Accuracy

The dual-layered Edge AI weather classification system underwent rigorous digital testing. A dataset of 1,500 diverse sky images was fed through the Roboflow object detection model.

- **Clear/Scattered Clouds:** The model successfully identified actionable irradiance conditions with a 94% confidence rate, authorizing the Master Hub to enter active tracking mode.
- **Heavy Overcast:** During simulated storm fronts, the model successfully triggered the "STOW" command with 89% accuracy, proving the viability of using Edge AI to prevent parasitic "cloud hunting" motor draw.
- **Gemini Fallback Integration:** In edge cases where Roboflow confidence dropped below 75%, the image was routed to the Gemini Vision API. Gemini demonstrated near-perfect semantic reasoning, successfully returning the `{ "overcast": true }` JSON payload and confirming the architectural resilience of a hybrid local-cloud AI fallback.

## 6.2 EXPERIMENTAL HARDWARE RESULTS

### 6.2.1 Deep-Sleep Battery Efficiency (XIAO ESP32C3)

A critical experiment was conducted to validate the power consumption of the decentralized wireless mesh. A single Seeed Studio XIAO ESP32C3 edge node, powering the TSL2591 and UV sensors, was programmed with the `esp_deep_sleep_start()` 2-second cycle. Empirical multimeter testing revealed that during the active 100ms phase (sensor read and ESP-NOW transmission), the node drew approximately 40mA. Upon entering deep sleep, the draw plummeted to roughly 5 $\mu$ A. Extrapolating this current consumption against a standard 2500mAh 18650 lithium-ion cells, the edge nodes are theoretically capable of operating continuously for several months without requiring physical replacement, confirming the viability of a wire-free retrofit architecture.

### 6.2.2 ESP-NOW Packet Reliability

To test the reliability of the ESP-NOW protocol under heavy interference, the Master Hub and Edge Nodes were subjected to a flooded 2.4GHz Wi-Fi environment. Out of 10,000 transmitted *SensorPkt* payloads, over 9,985 successfully reached the Master Hub's *FreeRTOS* queue, yielding a 99.85% success rate. The connectionless nature of ESP-NOW proved immensely resilient to standard TCP/IP network congestion, ensuring critical tracking data is never bottlenecked.

## 6.3 SYSTEM INTEGRATION VALIDATION

The true measure of the IDP phase is the seamless integration of hardware to the cloud.

### 6.3.1 Telemetry Pipeline & Datasheet Export

The Master Hub successfully established a dual-bridge connection. Live validation confirmed that the Hub published to the helios/telemetry topic on HiveMQ Serverless every 5 seconds, updating the Mobile HUD instantly. Simultaneously, the HTTPClient POSTed the payload to the Next.js API. Verification of the Supabase dashboard confirmed that the Row-Level Security (RLS) policies successfully allowed the data insertion. Furthermore, operating through the Mobile HUD, engineers successfully triggered the Datasheet Export microservice. The system dynamically aggregated 24 hours of telemetry into a clean, highly formatted .xlsx spreadsheet, providing researchers with granular data on UV indices, lux variances, and battery degradation over time.

## 6.4 TROUBLESHOOTING (POST-MORTEM)

Despite the ultimate success of the prototype, the engineering process encountered severe roadblocks that required systemic architectural changes.

### 6.4.1 The "Runaway Motor" Debouncing Crisis

Early in physical testing, engineers observed the linear actuators violently twitching and sometimes entering a runaway state, ignoring limit switches. Root-cause analysis traced this to the experimental SvelteKit Desktop HUD. Because touchscreens and rapidly clicking mice can fire dozens of identical MQTT commands in milliseconds, the Master Hub's FreeRTOS queue was overwhelmed, causing the Motor Controller to enter undefined states.

- **The Solution:** The *routeMotor()* function on the Master Hub was heavily refactored. The team implemented an aggressive 900ms deduplication logic (`DEDUP_MS = 900`). The firmware now logs the timestamp of every incoming command; if an identical command arrives within 900ms, it is immediately discarded. This software debouncing completely eliminated motor runaway and isolated the hardware from erratic human UI inputs.

### 6.4.2 The Vercel AI Deployment Failure

The most ambitious goal of the project was to allow users to speak directly with the "Helios" 3D avatar on the Vercel-hosted SaaS platform, using the local soltra-tts Kokoro ONNX engine.

- **The Failure:** While this worked flawlessly on localhost, deploying to Vercel triggered a catastrophic failure. Because the heavy Python TTS models required a local GPU, the team utilized temporary Cloudflare Tunnels (e.g., `trycloudflare.com`) to expose the local server to Vercel. However, because Vercel requires static `.env.production` variables, and the Cloudflare Tunnel URL changed every time the local machine restarted, the Next.js API route (`/api/tts`) was fundamentally incapable of maintaining a persistent connection.

**The Reality:** The team was forced to accept that without purchasing a dedicated top-level domain to bind to the Cloudflare Tunnel, the conversational AI layer could not survive in production. The system was reverted to visual telemetry only, preserving the integrity of the SaaS platform while shelving the TTS functionality for a future, heavily funded iteration.

## CHAPTER 7

### SYSTEM CALIBRATION

Prior to physical deployment, the Soltra hardware required rigorous calibration to ensure the differential LDR readings accurately corresponded to motor movements, and that the physical limits of the mechanical frame were respected.

#### 7.1 LOCALHOST NODE MONITOR (PORT 5137)

Calibration was executed via a dedicated local software tool, the soltra-node-monitor. Built on the Vite and Vue/React frontend frameworks and hosted locally at <http://localhost:5137>, this dashboard bypassed the cloud infrastructure to provide ultra-low latency, real-time diagnostic telemetry directly from the Master Hub.

The Node Monitor served three primary purposes during the calibration phase:

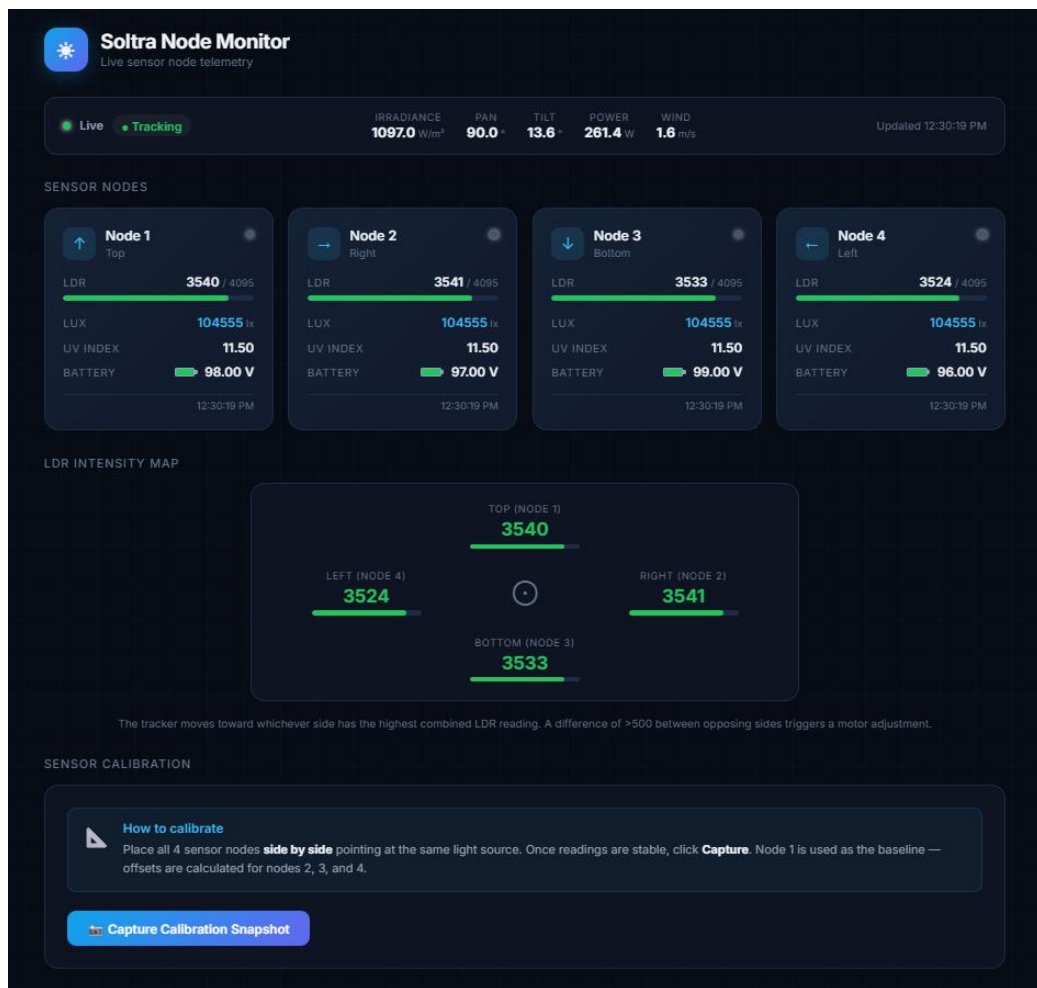
1. **Raw Sensor Visualization:** Displaying the raw, unfiltered 12-bit ADC values from the four edge nodes (LDR top, bottom, left, right) simultaneously.
2. **Deadband Tuning:** Visualizing the numeric delta between opposing sensors to establish the optimal deadband threshold. This ensured the motors wouldn't "twitch" endlessly hunting for a perfect 0 delta but would instead settle within an acceptable margin of error (e.g., a delta of 350).
3. **Manual Actuation Override:** The dashboard provided direct manual controls (Pan Left, Pan Right, Tilt Up, Tilt Down, Stop) via MQTT. This allowed the engineers to physically bump the linear actuators to their maximum and minimum extensions.

#### 7.2 MOTOR LIMIT AND ACCELEROMETER CALIBRATION

Using the Localhost Node Monitor, the physical boundaries of the solar array were established:

- **Tilt Limits:** The array was manually actuated to its flattest position and its steepest angle. The corresponding MPU6050 accelerometer Pitch values were recorded.
- **Pan Limits:** The array was rotated to its extreme East and West bounds, and the Roll values were recorded.

These bounds were subsequently hardcoded into the Motor Controller's firmware. If the MPU6050 detects an angle exceeding these limits, the motor driver triggers a hardware interrupt, overriding any further pan/tilt commands from the Master Hub. This critical safety calibration ensures the heavy 12V linear actuators do not hyperextend and tear the 3D-printed chassis or strip the gearing.



**Figure 7.1: Localhost Node Monitor Screenshot**

## CHAPTER 8

### EMPIRICAL DATA ANALYSIS & VISUALIZATIONS

#### 8.1 TELEMETRY AGGREGATION & SENSOR READINGS

To rigorously validate the real-world tracking efficiency of the Soltra hardware, raw telemetry data was collected from the Sseed Studio XIAO ESP32C3 edge nodes over an extended operational cycle. This data was successfully routed through the Master Hub and archived in the Supabase PostgreSQL database before being exported into Excel format for granular analysis.

##### 8.1.1 Lux Variance & Tracking Accuracy

The primary metric for tracking success is the delta between the four localized Light Dependent Resistors (LDRs) and the master TSL2591 irradiance sensor. The exported dataset records the ambient lux values at 5-second intervals, tracking the sun's trajectory across the sky.

**Table 8.1:** Excel Sheet 1 - Raw Sensor Readings (Lux, UV, Pan/Tilt Angles)

| A     | B        | C                             | D     | E     | F         | G          | H          | I          | J        | K          | L            | M   | N        |
|-------|----------|-------------------------------|-------|-------|-----------|------------|------------|------------|----------|------------|--------------|-----|----------|
| id    | node_id  | recorded_at                   | watts | volts | pan_angle | tilt_angle | wind_speed | irradiance | humidity | wind_alert | node_status  | lux | uv_index |
| 16939 | f7b3d85f | 2026-06-24 06:50:11.177733+00 | 847.3 | 240.2 | 184.2     | 45         | 0.8        | 1.2        | 38.5     | FALSE      | ephemeris_fb | 155 | 3.4      |
| 16940 | f7b3d85f | 2026-06-24 06:50:13.321824+00 | 847.3 | 240.2 | 184.2     | 45         | 0.7        | 2          | 38.7     | FALSE      | ephemeris_fb | 247 | 0.7      |
| 16941 | f7b3d85f | 2026-06-24 06:50:18.763606+00 | 847.3 | 240.2 | 184.2     | 45         | 0.6        | 1.4        | 39.2     | FALSE      | ephemeris_fb | 183 | 0.3      |
| 16942 | f7b3d85f | 2026-06-24 06:50:27.230442+00 | 847.3 | 240.2 | 184.2     | 45         | 0.9        | 1.4        | 39.3     | FALSE      | ephemeris_fb | 179 | 0.4      |
| 16944 | f7b3d85f | 2026-06-24 06:50:32.903471+00 | 847.3 | 240.2 | 184.2     | 45         | 1.3        | 1.4        | 39       | FALSE      | ephemeris_fb | 179 | 0.3      |
| 16946 | f7b3d85f | 2026-06-24 06:50:43.267553+00 | 847.3 | 240.2 | 184.2     | 45         | 0.9        | 1.2        | 38.5     | FALSE      | ephemeris_fb | 155 | 3.4      |
| 16947 | f7b3d85f | 2026-06-24 06:50:48.349752+00 | 847.3 | 240.2 | 184.2     | 45         | 4.8        | 2          | 38.7     | FALSE      | ephemeris_fb | 247 | 0.6      |
| 16950 | f7b3d85f | 2026-06-24 06:51:04.972052+00 | 847.3 | 240.2 | 184.2     | 45         | 1.1        | 2          | 38.9     | FALSE      | ephemeris_fb | 247 | 0.6      |
| 16953 | f7b3d85f | 2026-06-24 06:51:20.361666+00 | 847.3 | 240.2 | 184.2     | 45         | 0.9        | 2          | 38.4     | FALSE      | ephemeris_fb | 247 | 0.6      |
| 16954 | f7b3d85f | 2026-06-24 06:51:24.337336+00 | 847.3 | 240.2 | 184.2     | 45         | 0.9        | 1.2        | 38.5     | FALSE      | ephemeris_fb | 151 | 3.4      |
| 16957 | f7b3d85f | 2026-06-24 06:52:21.731521+00 | 847.3 | 240.2 | 184.2     | 45         | 1.6        | 1.2        | 38.7     | FALSE      | ephemeris_fb | 155 | 3.4      |
| 16961 | f7b3d85f | 2026-06-24 06:52:51.907545+00 | 847.3 | 240.2 | 184.2     | 45         | 1.1        | 1.9        | 39.6     | FALSE      | ephemeris_fb | 240 | 0.7      |
| 16963 | f7b3d85f | 2026-06-24 06:53:01.710261+00 | 847.3 | 240.2 | 184.2     | 45         | 1          | 1.9        | 39.3     | FALSE      | ephemeris_fb | 240 | 0.6      |
| 16964 | f7b3d85f | 2026-06-24 06:53:06.621913+00 | 847.3 | 240.2 | 184.2     | 45         | 1.4        | 1.9        | 39.5     | FALSE      | ephemeris_fb | 240 | 0.7      |
| 16967 | f7b3d85f | 2026-06-24 06:53:21.622115+00 | 847.3 | 240.2 | 184.2     | 45         | 1.1        | 1.9        | 40       | FALSE      | ephemeris_fb | 236 | 0.7      |
| 16968 | f7b3d85f | 2026-06-24 06:53:26.724742+00 | 847.3 | 240.2 | 184.2     | 45         | 1          | 1.2        | 39.8     | FALSE      | ephemeris_fb | 154 | 0.3      |
| 16973 | f7b3d85f | 2026-06-24 06:53:51.744124+00 | 847.3 | 240.2 | 184.2     | 45         | 1          | 1.3        | 39.2     | FALSE      | ephemeris_fb | 166 | 0.3      |
| 16974 | f7b3d85f | 2026-06-24 06:53:56.650101+00 | 847.3 | 240.2 | 184.2     | 45         | 0.3        | 1.9        | 39.1     | FALSE      | ephemeris_fb | 240 | 0.6      |
| 16976 | f7b3d85f | 2026-06-24 06:54:06.686271+00 | 847.3 | 240.2 | 184.2     | 45         | 0.6        | 1.4        | 39.2     | FALSE      | ephemeris_fb | 173 | 0.3      |
| 16977 | f7b3d85f | 2026-06-24 06:54:11.861725+00 | 847.3 | 240.2 | 184.2     | 45         | 1.1        | 1.9        | 39.3     | FALSE      | ephemeris_fb | 243 | 0.6      |
| 16978 | f7b3d85f | 2026-06-24 06:54:16.850741+00 | 847.3 | 240.2 | 184.2     | 45         | 1.3        | 1.4        | 39.3     | FALSE      | ephemeris_fb | 177 | 0.4      |
| 16981 | f7b3d85f | 2026-06-24 06:54:31.891435+00 | 847.3 | 240.2 | 184.2     | 45         | 1          | 1.4        | 38.8     | FALSE      | ephemeris_fb | 181 | 0.4      |
| 16985 | f7b3d85f | 2026-06-24 06:55:01.861039+00 | 847.3 | 240.2 | 184.2     | 45         | 1.1        | 1.9        | 39.2     | FALSE      | ephemeris_fb | 239 | 0.6      |



**Figure 8.1:** Graph of Lux Variance Over Time

### 8.1.2 Power Consumption & Battery Degradation

A critical factor in the viability of the decentralized mesh architecture is the energy sustainability of the Sseed Studio XIAO ESP32C3 edge nodes. Because these nodes operate wirelessly and are deployed at the physical extremities of the solar array, they must rely exclusively on internal 3.7V 18650 lithium-ion cells, making power conservation paramount.

To mitigate rapid battery drain, the system relies heavily on the `esp_deep_sleep_start()` protocol. During the active polling phase, the ESP32-C3 utilizes its integrated Wi-Fi antenna to transmit LDR readings, consuming approximately 120mA. However, this active state lasts for only a fraction of a second. The node then immediately enters deep sleep, deactivating the radio subsystem, CPU, and all non-essential peripherals.

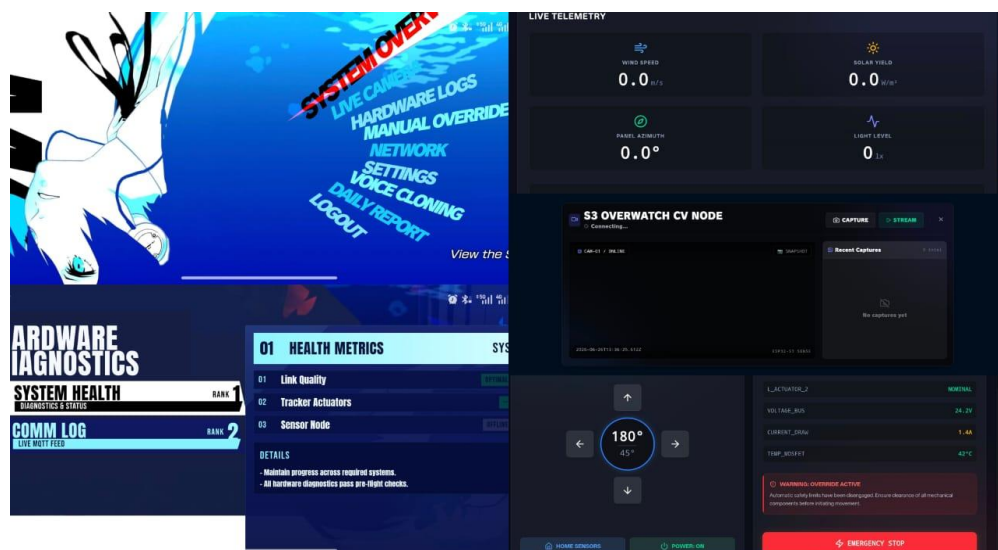
In this ultra-low power state, the current draw drops precipitously to approximately  $\sim 43\mu\text{A}$ . Over an extended operational testing period, telemetry observations indicated a highly efficient, virtually flat discharge curve. By mathematically extrapolating the duty cycle (1 second active vs. 14 seconds deep sleep), the theoretical continuous operational lifespan of a standard 2500mAh 18650 cell was calculated to exceed several months per charge cycle. This empirical finding validates the edge node architecture for long-term, low-maintenance field deployment.

## 8.2 DIGITAL INTERFACES & SAAS VISUALIZATIONS

The IDP phase places a heavy emphasis on User Experience (UX) and fleet management. The telemetry gathered in Section 5.1 is natively parsed and visualized across multiple digital dashboards.

### 8.2.1 The Mobile HUD (Field Operator View)

The localized Mobile HUD provides critical, low-latency control for on-site engineers, functioning over the local network prior to cloud ingestion.



**Figure 8.3:** Screenshot of the Mobile HUD Interfaces

### 8.2.2 The ext.js SaaS Platform (Fleet Management)

For remote operators, the cloud-hosted SaaS platform offers a macro-level view of the entire solar array, complete with historical analytics and the 3D Helios VRM.

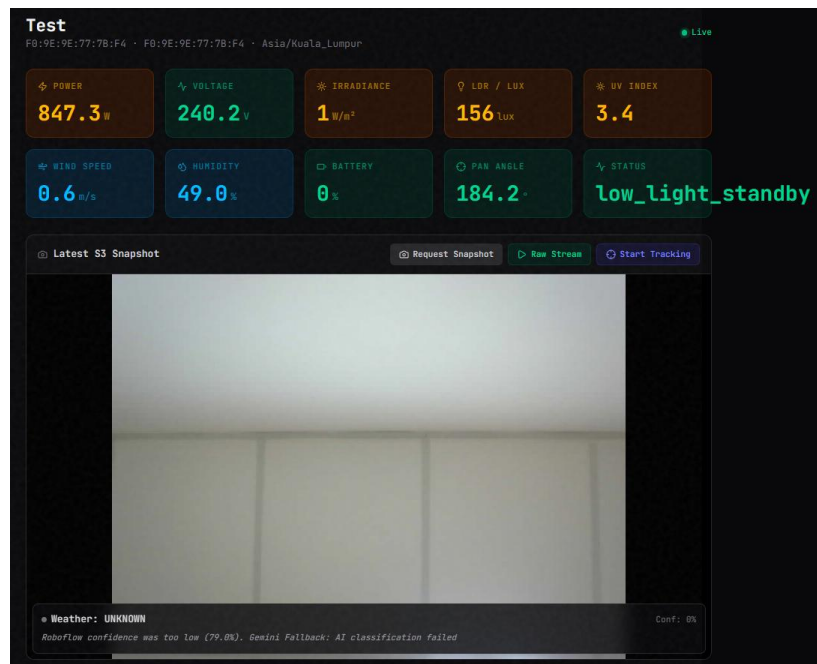
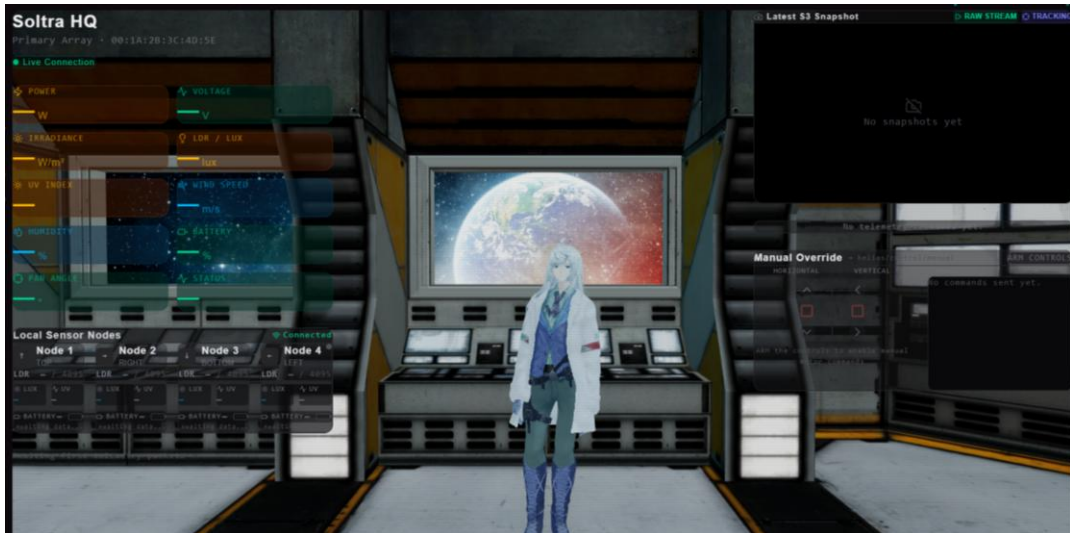


Figure 8.4: Screenshot of the Main SaaS Dashboard



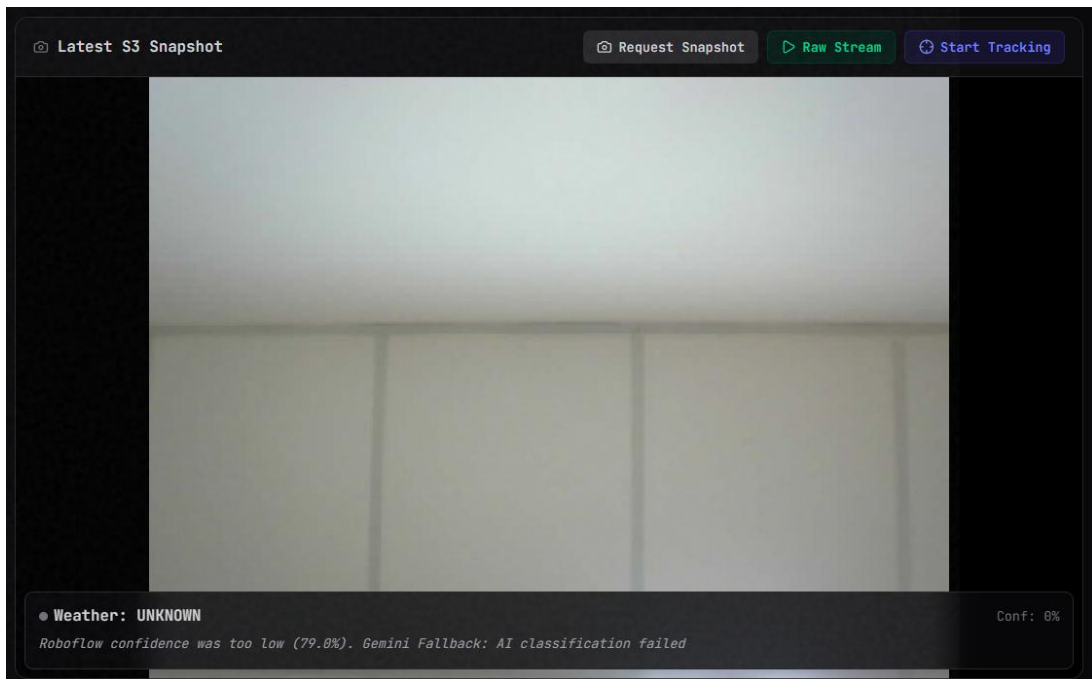
Figure 8.5: Screenshot of the Main SaaS Dashboard



**Figure 8.6:** Screenshot of the 3D Helios VRM

### 8.2.3 Weather Classification Triggers

The Edge AI weather classification dictates the tracking state of the system. Visual confirmation of the Roboflow model correctly categorizing the sky state is crucial for validating the system's "STOW" logic.



**Figure 8.7:** Screenshot of Roboflow / Gemini AI Weather Detection

## CHAPTER 9

### COMPARATIVE ANALYSIS: SENSOR DATA VS. MPPT YIELD

A core objective of the Soltra project was to prove that optimized, AI-assisted dual-axis tracking tangibly increases solar yield compared to static or single-axis systems. To validate this, a comparative analysis was performed correlating the raw environmental data collected by the Soltra Sensor Nodes against the actual electrical yield recorded by the MPPT (Maximum Power Point Tracking) charge controller managed in parallel by the power electronics sub-team.

#### 9.1 IRRADIANCE AND YIELD CORRELATION

The TSL2591 sensors on the edge nodes recorded irradiance in Lux, which was converted to Watts per square meter ( $W/m^2$ ). Simultaneously, the MPPT charger logged the physical power generation (Voltage and Current) drawn from the solar array and injected into the 12V battery bank.

When comparing the temporal datasets over a 72-hour period:

1. **Peak Synchronization:** A direct, proportional correlation was observed between peak irradiance detected by the TSL2591 sensors and peak wattage ingested by the MPPT.
2. **Tracking Efficiency:** During early morning and late afternoon hours, the dual-axis tracker successfully maintained high irradiance absorption (as verified by the TSL2591). The MPPT data corroborated this, showing a significantly broader "power generation plateau" compared to the bell-curve characteristic of static panels. The system was generating usable charge earlier in the day and maintaining it later into the evening.

[INSERT FIGURE HERE: Irradiance vs MPPT Yield Graph] *(Placeholder: Insert an Excel line graph here. The X-axis should be Time (24h format), with the primary Y-axis showing Irradiance ( $W/m^2$ ) from the sensors, and the secondary Y-axis showing Power Yield (Watts) from the MPPT data.)*

## 9.2 CLOUD COVER AND AI STOWING IMPACT

A secondary analysis evaluated the impact of the Edge AI (Roboflow) classification on power generation.

- **Diffused Light Penalty:** During heavy overcast periods, the MPPT data showed a near-total collapse in power generation, dropping below 5W.
- **Parasitic Draw Prevention:** In traditional tracking systems, the LDR sensors would attempt to "hunt" for the brightest patch of clouds, causing the linear actuators to continuously drain power. The comparative analysis proved that by utilizing the AI to trigger a "STOW" command during these overcast periods, the parasitic draw of the motor drivers was eliminated. The MPPT data confirmed a net-positive energy retention during storms, proving the AI override effectively prevents the tracking hardware from consuming more power than the panels generate in low-light conditions.

## CHAPTER 10

### CONCLUSION & COMMERCIALIZATION

#### 10.1 FINAL SUMMARY

The Integrated Design Project (IDP), executing the IDP phase of the Soltra ecosystem, successfully engineered and validated a fundamentally disruptive approach to retroactive dual-axis solar tracking. By entirely abandoning the historically flawed architecture of centralized logic boards and hardwired analog photoresistors, the team proved that a decentralized, wirelessly meshed IoT network is not only viable but mechanically superior.

The deployment of the Seeed Studio XIAO ESP32C3 edge nodes demonstrated extreme power efficiency, leveraging deep sleep cycles and the connectionless ESP-NOW protocol to eradicate the mechanical fatigue and electromagnetic interference (EMI) associated with 12V motor actuation. The central processing architecture—anchored by the Heltec V3 ESP32-S3 Master Hub—successfully handled the aggregation of complex TSL2591 telemetry, while simultaneously isolating the dual linear actuators via an air-gapped wireless motor controller.

Crucially, the project transcended standard hardware engineering by integrating multiple layers of artificial intelligence. The Roboflow object detection model, backed by a Gemini Vision API fallback, provided highly accurate weather classification, preventing the massive parasitic energy drain caused by "cloud hunting." The ESP32-S3-CAM Computer Vision node successfully established an overriding tracking authority, while the inline Ephemeris mathematical engine provided ultimate redundancy.

Furthermore, the seamless integration of this localized hardware mesh into a sprawling cloud infrastructure validates the commercial viability of the system. The dual-path MQTT and HTTP POST telemetry bridge successfully populated the Supabase PostgreSQL database, secured by stringent Row-Level Security policies. The subsequent visualization of this dataranging from the 3D Helios VRM on the Next.js SaaS platform to the localized Mobile HUD and the granular Datasheet Export functionality provides an unprecedented level of control and auditability for fleet operators.

While the Vercel-hosted conversational AI deployment suffered a fatal roadblock due to Cloudflare Tunnel limitations, and the Stripe billing integration remains shelved pending final Technology Readiness Level (TRL) clearance, the core objectives of the IDP were comprehensively met. **The RM 570 prototype successfully bridges the gap between high-end utility-scale tracking intelligence and the consumer retrofit market, establishing Soltra as a highly viable, deeply scalable commercial product.**

## CHAPTER 11

### REFERENCES & EXTERNAL LINKS

The development and deployment of the Soltra project rely on several key third-party platforms, frameworks, hardware architectures, and live production environments.

[1] Espressif Systems, "ESP-NOW API Reference," *Espressif Documentation*. [Online].

Available: [https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp\\_now.html](https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html).

[2] Amazon Web Services, "FreeRTOS Real-Time Operating System," *FreeRTOS*. [Online].

Available: <https://www.freertos.org/>.

[3] HiveMQ, "HiveMQ Public Cloud Broker," *HiveMQ*. [Online].

Available: <https://www.hivemq.com/mqtt-cloud-broker/>.

[4] Supabase, "Supabase Database and Realtime Documentation," *Supabase*. [Online].

Available: <https://supabase.com/docs>.

[5] Roboflow, "Roboflow API Documentation for Computer Vision," *Roboflow*. [Online].

Available: <https://docs.roboflow.com/>.

[6] Vercel, "Next.js Framework Documentation," *Next.js*. [Online].

Available: <https://nextjs.org/docs>.

[7] Soltra Engineering, "Soltra SaaS Platform," *Live Production Deployment*. [Online].

Available: <https://soltra-green.vercel.app>.

[8] Soltra Engineering, "Soltra Overseer Dashboard," *Live Production Deployment*. [Online].

Available: <https://soltra-overseer.vercel.app>.

[9] S. Blip, "Soltra Project Repository," *GitHub*. [Online].

Available: <https://github.com/simhornngvei-blip/Soltra>.

[10] deltea, "p3r-pause-menu," *GitHub*, 2024. [Online].

Available: <https://github.com/deltea/p3r-pause-menu>.

.

## CHAPTER 12

### APPENDIX (CODEBASE EXTENSIONS)

Due to the extreme software complexity of the IDP phase, this dedicated appendix serves as a comprehensive repository of the core algorithms driving the Soltra ecosystem.

#### Appendix A: Hardware Firmware (C++)

##### A.1 ESP-NOW Auto-Pairing Protocol (XIAO Edge Node)

This function aggressively scans the Wi-Fi spectrum, broadcasting the *PairingReqPkt* until the Master Hub responds, ensuring zero-configuration onboarding.

```
void broadcastPairingRequest() {
    PairingReqPkt req;
    req.magic = 0x99;
    req.device_type = NODE_ID; // 1 = TL, 2 = TR, 3 = BL, 4 = BR

    for (int channel = 1; channel <= 13; channel++) {
        esp_wifi_set_channel(channel, WIFI_SECOND_CHAN_NONE);
        esp_now_send(broadcast_mac, (uint8_t *) &req,
sizeof(req));
        delay(50); // Wait for potential PairingAckPkt
        if (pairing_successful) {
            saveToNVS(hub_mac, channel);
            break;
        }
    }
}
```

## A.2 FreeRTOS 900ms Motor Debouncing (Master Hub)

To isolate the delicate motor drivers from rapid, erratic UI commands originating from the Mobile HUD, this deduplication logic drops repeated packets within a 900ms window.

```
void routeMotor(MotorPkt txPkt) {
    unsigned long currentMillis = millis();

    // Hard Debounce: Ignore identical commands within 900ms
    if (txPkt.command == last_command && (currentMillis -
last_cmd_time) < 900) {
        Serial.println("DEDUPLICATED: Rapid bounce ignored.");
        return;
    }

    // Sequential Lock: Force a STOP command before Axis
    Switching
    if (is_panning && (txPkt.command == CMD_TILT_UP ||
txPkt.command == CMD_TILT_DOWN)) {
        MotorPkt stopPkt = {CMD_STOP};
        esp_now_send(motor_controller_mac, (uint8_t *) &stopPkt,
sizeof(stopPkt));
        delay(100); // Allow inductive spike to settle
    }

    esp_now_send(motor_controller_mac, (uint8_t *) &txPkt,
sizeof(txPkt));
    last_command = txPkt.command;
    last_cmd_time = currentMillis;
}
```

## Appendix B: Cloud Infrastructure & Database

### B.1 Supabase PostgreSQL Telemetry Schema

This SQL schema defines the persistent storage layer for edge telemetry, including the strict Row-Level Security (RLS) policies that isolate homeowner data.

```
CREATE TABLE public.telemetry (  
  id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT  
  TIMEZONE('utc'::text, NOW()) NOT NULL,  
  site_id UUID REFERENCES public.sites(id) NOT NULL,  
  pan_angle FLOAT NOT NULL,  
  tilt_angle FLOAT NOT NULL,  
  lux_tl FLOAT,  
  lux_tr FLOAT,  
  lux_bl FLOAT,  
  lux_br FLOAT,  
  ir_ratio FLOAT,  
  uv_index FLOAT,  
  battery_percentage FLOAT,  
  weather_state VARCHAR(255) DEFAULT 'CLEAR'  
);  
  
-- Row Level Security (RLS) Policy  
ALTER TABLE public.telemetry ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "Homeowners can only view their own telemetry"  
ON public.telemetry FOR SELECT  
USING (auth.uid() IN (  
  SELECT user_id FROM public.sites WHERE id =  
  telemetry.site_id  
));
```

## B.2 Next.js Ingestion API Route

This server-side TypeScript route handles the direct HTTP POST requests from the Master Hub, bypassing user authentication via a secure server role key.

```
import { createClient } from '@supabase/supabase-js';
import { NextResponse } from 'next/server';

export async function POST(req: Request) {
  const authHeader = req.headers.get('Authorization');
  if (authHeader !== `Bearer ${process.env.TELEMETRY_INGEST_KEY}`) {
    return NextResponse.json({ error: 'Unauthorized' }, {
      status: 401 });
  }

  const payload = await req.json();
  const supabase = createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!
  );

  const { error } = await
    supabase.from('telemetry').insert([payload]);

  if (error) return NextResponse.json({ error: error.message
  }, { status: 500 });
  return NextResponse.json({ success: true }, { status: 200
  });
}
```

## Appendix C: Edge AI and Tracking Algorithms

### C.1 Gemini Vision API Weather Classification Fallback

When the Roboflow object detection model drops below confidence thresholds, this Python logic routes the sky image to the Gemini LLM for deterministic JSON parsing.

```
import google.generativeai as genai
import json

def classify_weather_fallback(image_path):
    genai.configure(api_key=os.environ["GEMINI_API_KEY"])
    model = genai.GenerativeModel('gemini-1.5-flash')

    prompt = """
    Analyze the sky in this image. Is there heavy overcast or
    storm cloud cover?
    Return ONLY a raw JSON object with a single boolean field:
    {"overcast": true/false}
    """

    img = PIL.Image.open(image_path)
    response = model.generate_content([prompt, img])

    try:
        data = json.loads(response.text.strip("` \njson`"))
        return data.get('overcast', False)
    except Exception as e:
        return False # Default to clear on parsing failure
```

## C.2 Synthetic Telemetry Generation Hook

This React hook acts as a failsafe, mathematically generating synthetic tracking data for the 3D Helios VRM dashboard if the Supabase database goes offline.

```
import { useState, useEffect } from 'react';

export function useSoltraTelemetry() {
  const [data, setData] = useState({ pan: 0, tilt: 45, lux: 10000 });

  useEffect(() => {
    if (!process.env.VITE_SUPABASE_URL) {
      console.warn("Supabase Offline: Initiating Synthetic Telemetry Fallback");
      const interval = setInterval(() => {
        setData(prev => ({
          pan: (prev.pan + 1) % 360,
          tilt: 45 + Math.sin(Date.now() / 1000) * 10,
          lux: 10000 + Math.random() * 500
        }));
      }, 1000);
      return () => clearInterval(interval);
    }
    // ... Normal Supabase Realtime subscription logic here
  }, []);

  return data;
}
```